



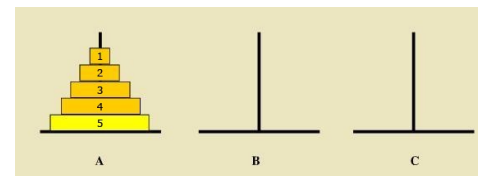
第一章 基础语法

南开大学软件学院

王 斌辉

计算思维 *Computational Thinking*

- **逻辑思维**: 推理和演绎,数学为代表, $A \rightarrow B$ $B \rightarrow C$ $A \rightarrow C$
- **实证思维**: 实验和验证,物理学为代表, 重力, 引力波 $\leftarrow$ 实验
- **工程思维**: 目标导向, 以用户为中心, 约束条件下最优化、系统化解决问题, 迭代
- **计算思维**: **Abstraction (抽象)** 和 **Automation (自动化)** 是计算思维的两大核心特征
 - 计算思维是与形式化问题及其解决方案相关的思维过程, 其解决问题的表示形式应该能有效地被信息处理代理执行 (CMU周以真 (Jeannette M. Wing) 2010)
 - 运用计算机科学的基础概念去求解问题、设计系统和理解人类的行为
 - 是递归思维, 是并行处理, 是把代码译成数据又能把数据译成代码。如汉诺塔递归
 - 是通过约简、嵌入、转化和仿真等方法, 把一个困难问题阐释成一个如何解决的问题
 - 是一种采用抽象和分解来控制庞杂的任务或进行巨大复杂系统设计的方法
 - 对其它学科的影响。统计学、生物学、社会科学.....



The Zen of Python

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than **right** now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

Python Programming

优美胜于丑陋（Python 以编写优美的代码为目标）

明了胜于晦涩（优美的代码应当是明了的，命名规范，风格相似）

简洁胜于复杂（优美的代码应当是简洁的，不要有复杂的内部实现）

复杂胜于凌乱（如果复杂不可避免，那代码间也不能有难懂的关系，要保持接口简洁）

扁平胜于嵌套（优美的代码应当是扁平的，不能有太多的嵌套）

间隔胜于紧凑（优美的代码有适当的间隔，不要奢望一行代码解决问题）

可读性很重要（优美的代码是可读的）

即便假借特例的实用性之名，也不可违背这些规则（这些规则至高无上）

不要包容所有错误，除非你确定需要这样做（精准地捕获异常，不写 `except:pass` 风格的代码）

当存在多种可能，不要尝试去猜测

而是尽量找一种，最好是唯一一种明显的解决方案（如果不确定，就用穷举法）

虽然这并不容易，因为你不是 Python 之父（这里的 Dutch 是指 Guido）

做也许好过不做，但不假思索就动手还不如不做（动手之前要细思量）

如果你无法向人描述你的方案，那肯定不是一个好方案；反之亦然（方案测评标准）

命名空间是一种绝妙的理念，我们应当多加利用（倡导与号召）



Python解释器

■ CPython

- 官方版本的解释器。在命令行下运行python就是启动CPython解释器
- CPython是用C语言开发的，所以叫CPython
- CPython是使用最广的Python解释器

■ IPython

- IPython是基于CPython之上的一个交互式解释器，IPython只是在交互方式上有所增强，但是执行Python代码的功能和CPython是完全一样的。好比很多国产浏览器虽然外观不同，但内核其实都是调用了IE。
- CPython用>>>作为提示符，而IPython用In [序号]:作为提示符。



- **PyPy**

PyPy采用JIT技术，对Python代码进行动态编译（注意不是解释），所以可以显著提高Python代码的执行速度。

- **Jython**

运行在Java平台上的Python解释器，可以直接把Python代码编译成Java字节码执行。

- **IronPython**

IronPython和Jython类似，只不过IronPython是运行在微软.Net平台上的Python解释器，可以直接把Python代码编译成.Net的字节码。



Python的解释器很多，但使用最广泛的还是CPython。如果要和Java或.Net平台交互，最好的办法不是用Jython或IronPython，而是**通过网络调用来交互**，确保各程序之间的独立性。



输出 Hello world!

- **Python交互模式**

在命令行模式下敲命令`python/python3`，就看到类似如下的一堆文本输出，然后就进入到Python交互模式，它的提示符是`>>>`

- **命令行模式和Python交互模式**

- **编辑器和IDE**



内容的输入输出

■ 输出

- print()函数也可以接受多个字符串，用逗号“,” 隔开，就可以连成一串输出。
- print()会依次打印每个字符串，遇到逗号“,” 会输出一个空格

```
print('The quick brown fox' , 'jumps over' , 'the lazy dog')
```

The quick brown fox jumps over the lazy dog

■ 输入

- Python提供了input(), 可让用户输入字符串，并存放放到变量里



标识符

- 标识符：用于变量、函数、类、模块等的名称。
- 标识符有如下特定的规则
 - 区分大小写，如: sxt和SXT是不同的
 - 第一个字符必须是字母、下划线，其后的字符是:字母、数字、下划线
 - 不能使用关键字，如: if, or, while等
 - 以双下划线开头和结尾的名称通常有特殊含义，如: `__init__` 是类的初始函数。

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	async
break	except	in	raise	await



对象

- Python中一切皆对象
- 每个对象由 **标识(identity)**、**类型(type)**、**value (值)** 组成
 - 标识用于唯一标识对象,通常对应于对象在计算机内存中的地址。使用内置函数`id(obj)`可返回对象`obj`的标识
 - 类型用于表示对象存储的"数据"的类型。类型可以限制对象的取值范围以及可执行的操作。可使用`type(obj)`获得对象的所属类型
 - 值表示对象所存储的数据的信息。使用`print(obj)`可以直接打印出值
- **对象的本质: 一个内存块,拥有特定的值,支持特定类型的相关操作**

■ 内存中对象存储类比



■ 对象的引用

- 在Python中，变量也称为：对象的引用
- 变量存储的就是对象的地址
- 变量位于：栈内存（压栈出栈）
- 对象位于：堆内存
- Python是动态类型语言，变量不需要显式声明类型。根据变量引用的对象，Python解释器自动确定数据类型。
- Python是强类型语言，每个对象都有数据类型，只支持该类型支持的操作。如 `99 + " 88"`

```
a = 1
b = a
a = 3
print(a, b)    3 1
```


■ 变量赋值基础

— Python赋值

```
x = y = z = 99
```

```
x, y, z = 99, 88, 77
```

```
x, *y = 99, 88, 77    x = 99, 88, 77
```

```
print(y)
```

```
[88, 77]
```



■ 变量赋值基础

— Python赋值

```
a = 99
```

```
b = 88
```

```
b, a = a, a + b
```

```
print(a)
```

187

```
print(b)
```

99

■ 变量赋值进阶

— C语言赋值

int x = 99;

x	
location	0x2a7
value	99

int x = 88;

x	
location	0x2a7
value	88

int y = x;

x	
location	0x2a7
value	88

y	
location	0x2a9
value	88

■ 变量赋值进阶

— Python赋值

`x = 99`

Python Name
x

PyObject	
Type	Integer
Value	99
Reference Count	1

- ① Create a PyObject
- ② Set the typecode to integer for the PyObject
- ③ Set the value to 99 for the PyObject
- ④ Create a name called x
- ⑤ Point x to the new PyObject
- ⑥ Increase the refcount of the PyObject by 1

■ 变量赋值进阶

— Python赋值

`x = 99`

`x = 88`

- ① Creates a new PyObject
- ② Sets the typecode to integer for the PyObject
- ③ Sets the value to 88 for the PyObject
- ④ Points x to the new PyObject
- ⑤ Increases the refcount of the new PyObject by 1
- ⑥ Decreases the refcount of the old PyObject by 1

Python Name
x

PyObject	
Type	Integer
Value	99
Reference Count	0

PyObject	
Type	Integer
Value	88
Reference Count	1

■ 变量赋值进阶

— Python赋值

```
x = 99
```

```
x = 88
```

```
y = x
```

```
print(x, y)
```

```
88 88
```

Python Name
x

Python Name
y

Garbage Collection

PyObject	
Type	Integer
Value	99
Reference Count	0

PyObject	
Type	Integer
Value	88
Reference Count	2

■ 变量赋值进阶

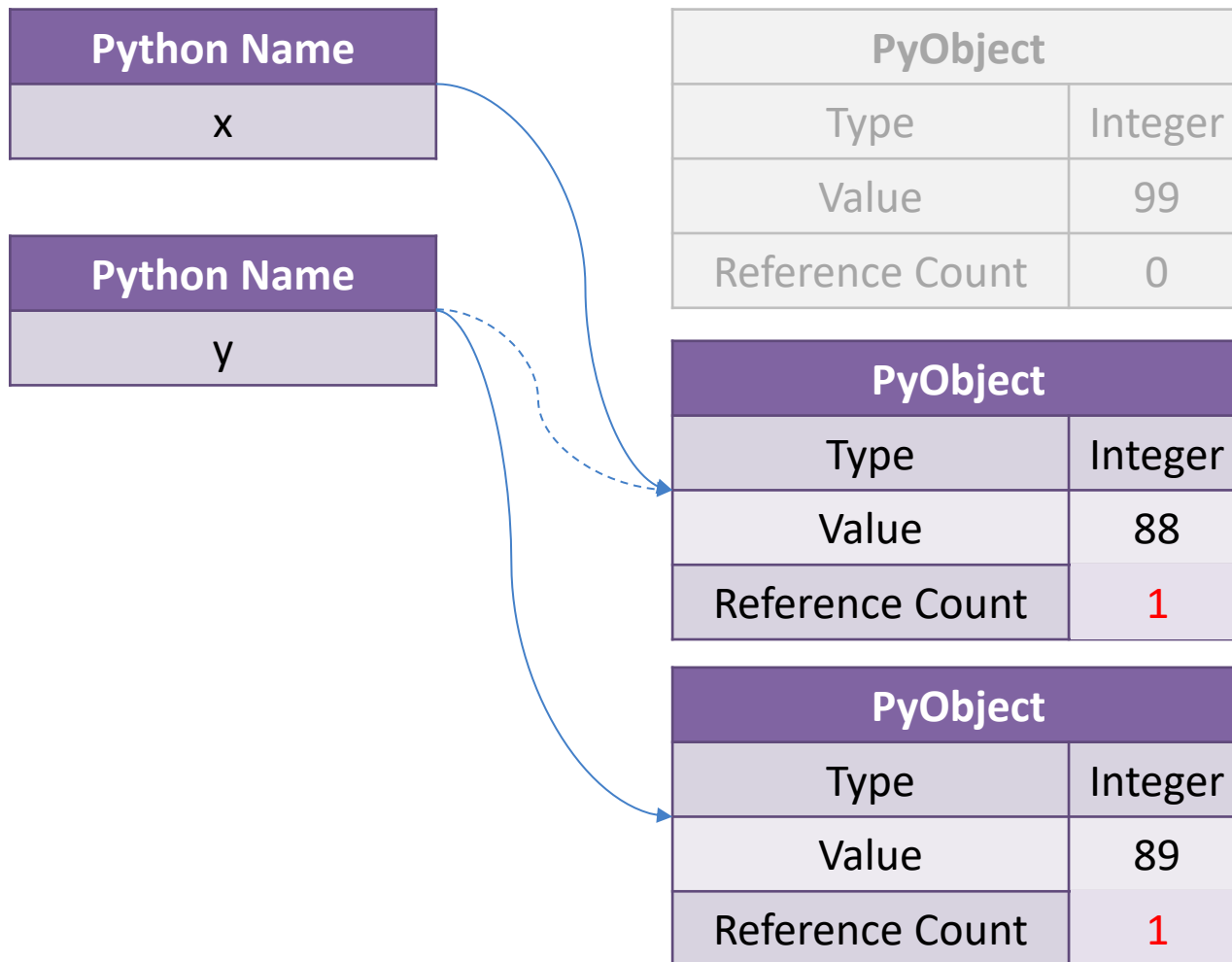
— Python赋值

```
x = 99
```

```
x = 88
```

```
y = x
```

```
y += 1
```



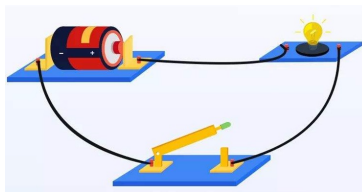


Python编码专题

```
#!/usr/bin/env python3  
# -*- coding: utf-8 -*-
```

二进制

- 1个元器件称为1比特 (Bit) 或1位, 8个元器件称为1字节 (Byte), 那么16个元器件就是2Byte, 32个就是4Byte, 以此类推:
 - 8×1024 个元器件就是1024Byte, 简写为1KB;
 - $8 \times 1024 \times 1024$ 个元器件就是1024KB, 简写为1MB;
 - $8 \times 1024 \times 1024 \times 1024$ 个元器件就是1024MB, 简写为1GB。



单位换算:

- 1Byte = 8 Bit
- 1KB = 1024Byte = 2^{10} Byte
- 1MB = 1024KB = 2^{20} Byte
- 1GB = 1024MB = 2^{30} Byte
- 1TB = 1024GB = 2^{40} Byte
- 1PB = 1024TB = 2^{50} Byte
- 1EB = 1024PB = 2^{60} Byte



十进制	二进制	符号	十进制	二进制	符号	十进制	二进制	符号	十进制	二进制	符号
0	0000 0000	NUL	32	0010 0000	[空格]	64	0100 0000	@	96	0110 0000	`
1	0000 0001	SOH	33	0010 0001	!	65	0100 0001	A	97	0110 0001	a
2	0000 0010	STX	34	0010 0010	"	66	0100 0010	B	98	0110 0010	b
3	0000 0011	ETX	35	0010 0011	#	67	0100 0011	C	99	0110 0011	c
4	0000 0100	EOT	36	0010 0100	\$	68	0100 0100	D	100	0110 0100	d
5	0000 0101	ENQ	37	0010 0101	%	69	0100 0101	E	101	0110 0101	e
6	0000 0110	ACK	38	0010 0110	&	70	0100 0110	F	102	0110 0110	f
7	0000 0111	BEL	39	0010 0111	`	71	0100 0111	G	103	0110 0111	g
8	0000 1000	BS	40	0010 1000	(	72	0100 1000	H	104	0110 1000	h
9	0000 1001	HT	41	0010 1001	)	73	0100 1001	I	105	0110 1001	i
10	0000 1010	LF	42	0010 1010	*	74	0100 1010	J	106	0110 1010	j
11	0000 1011	VT	43	0010 1011	+	75	0100 1011	K	107	0110 1011	k
12	0000 1100	FF	44	0010 1100	,	76	0100 1100	L	108	0110 1100	l
13	0000 1101	CR	45	0010 1101	-	77	0100 1101	M	109	0110 1101	m
14	0000 1110	SO	46	0010 1110	.	78	0100 1110	N	110	0110 1110	n
15	0000 1111	SI	47	0010 1111	/	79	0100 1111	O	111	0110 1111	o
16	0001 0000	DLE	48	0011 0000	0	80	0101 0000	P	112	0111 0000	p
17	0001 0001	DC1	49	0011 0001	1	81	0101 0001	Q	113	0111 0001	q
18	0001 0010	DC2	50	0011 0010	2	82	0101 0010	R	114	0111 0010	r
19	0001 0011	DC3	51	0011 0011	3	83	0101 0011	S	115	0111 0011	s
20	0001 0100	DC4	52	0011 0100	4	84	0101 0100	T	116	0111 0100	t
21	0001 0101	NAK	53	0011 0101	5	85	0101 0101	U	117	0111 0101	u
22	0001 0110	SYN	54	0011 0110	6	86	0101 0110	V	118	0111 0110	v
23	0001 0111	ETB	55	0011 0111	7	87	0101 0111	W	119	0111 0111	w
24	0001 1000	CAN	56	0011 1000	8	88	0101 1000	X	120	0111 1000	x
25	0001 1001	EM	57	0011 1001	9	89	0101 1001	Y	121	0111 1001	y
26	0001 1010	SUB	58	0011 1010	:	90	0101 1010	Z	122	0111 1010	z
27	0001 1011	ESC	59	0011 1011	;	91	0101 1011	[	123	0111 1011	{
28	0001 1100	FS	60	0011 1100	<	92	0101 1100	\	124	0111 1100	
29	0001 1101	GS	61	0011 1101	=	93	0101 1101	]	125	0111 1101	}
30	0001 1110	RS	62	0011 1110	>	94	0101 1110	^	126	0111 1110	~
31	0001 1111	US	63	0011 1111	?	95	0101 1111		127	0111 1111	DEL

字符集 (Character Set) 或者字符编码 (Character Encoding)

- **ASCII 编码**(American Standard Code for Information Interchange)
 - 标准 ASCII 编码共收录了 128 个字符，其中包含了 33 个控制字符（具有某些特殊功能但是无法显示的字符）和 95 个可显示字符。
- **GB2312编码**；
 - 要处理中文显然一个字节是不够的，至少需要两个字节
 - 日、韩、德、法...各有各国的标准，出现冲突，在多语言混合的文本中，产生**乱码**
- **Unicode字符集**
 - Unicode标准也在不断发展，但最常用的是UCS-16编码，用两个字节表示一个字符（如果要用到非常偏僻的字符，就需要4个字节）

如果统一成Unicode编码，乱码问题就此消失，但如果保存的文本大部分都是英文字符，使用Unicode编码，比ASCII编码几乎需要多一倍的存储空间，在存储和传输上便存在问题。如何解决这一问题呢？



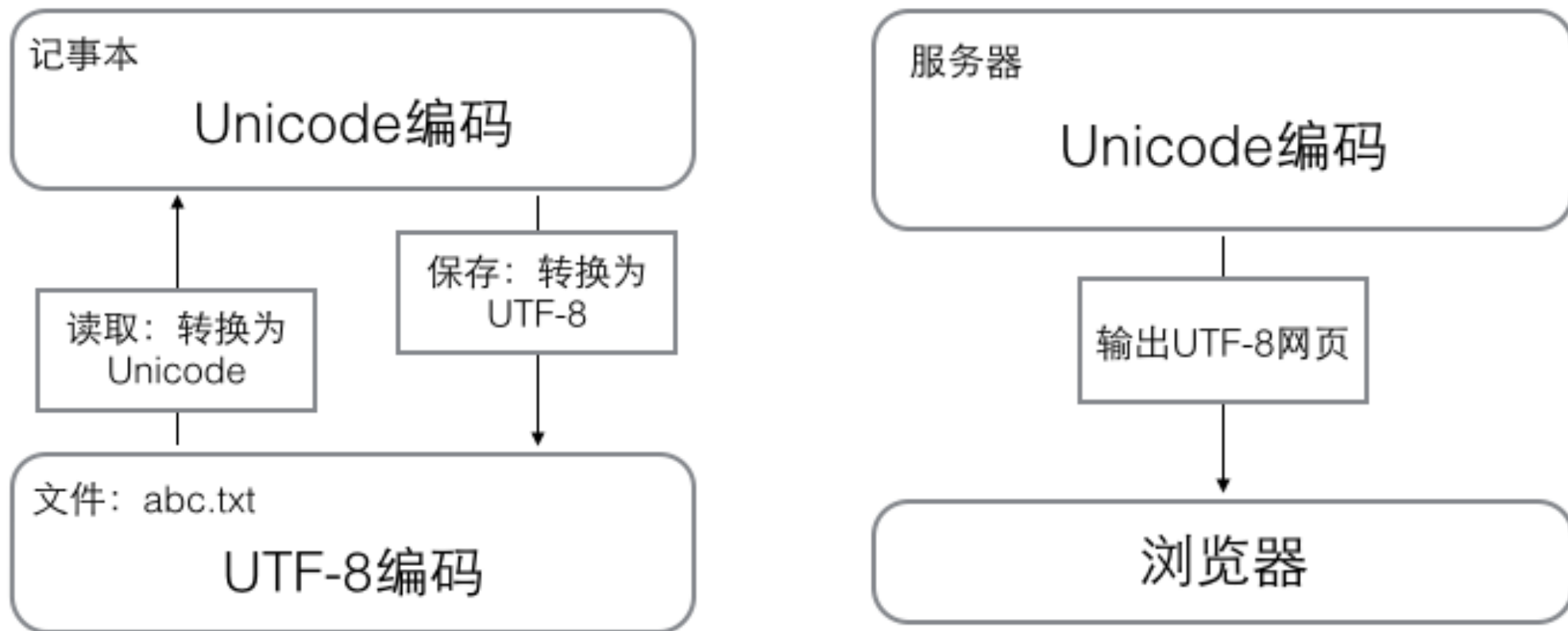


UTF-8编码

字符	ASCII	Unicode	UTF-8
A	01000001	00000000 01000001	01000001
中	x	01001110 00101101	11100100 10111000 10101101

UTF-8编码把一个Unicode字符，根据不同的数字大小编码成1-6个字节，常用的英文字母被编码成1个字节，汉字通常是3个字节，只有很生僻的字符才会被编码成4-6个字节。当要传输的文本包含大量英文字符，用UTF-8编码就能节省不少空间。

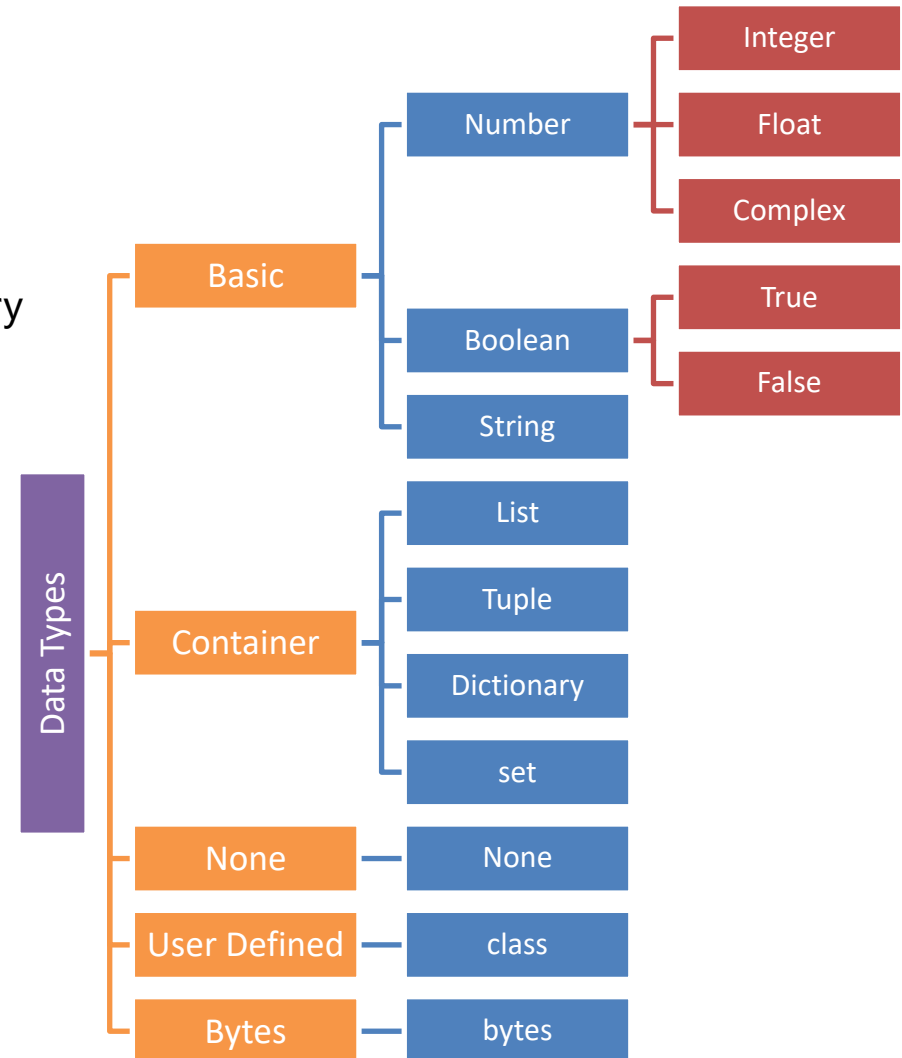
文件或者网页使用不同编码方式打开会产生乱码



■ Python数据类型

- Basic/Primitive – Int, Float, Complex, Bool, String
- Container/Collections – List, Tuple, Set, Dictionary
- None
- User Defined – class
- **bytes**

bytes is a new type in Python 3.x and is not available in Python 2.x.



Python数据类型

Data Type	Example
Integer	<code>a = 5</code>
Float	<code>a = 5.5</code>
Complex	<code>a = 5+7j</code>
Boolean	<code>a = True, a = False</code>
String	<code>a = "John", a = 'Steve'</code>
List	<code>a = [1,2,3,4,5]</code>
Tuple	<code>a = (1,2,3,4,5)</code>
Set	<code>a = {1,2,3,4,5}</code>
Dictionary	<code>a = {'name':'Steve', 'company':'apple'}</code>

python2	python3	备注
<code>x = 10000000000000L</code>	<code>x = 10000000000000</code>	python2中的十进制长整型在python3中被替换为十进制普通整数
<code>x = 0xFFFFFFFFFFFFL</code>	<code>x = 0xFFFFFFFFFFFF</code>	python2里的十六进制长整型在python3里被替换为十六进制的普通整数
<code>long(x)</code>	<code>int(x)</code>	python3没有long()
<code>type(x) is long</code>	<code>type(x) is int</code>	python3用int判断是否为整型
<code>isinstance(x, long)</code>	<code>isinstance(x, int)</code>	int检查整数类型



■ Python数据类型

- 可变对象(mutable object): 字典、列表、集合、自定义对象 (依赖用户定义的对象特征)
- 不可变对象(immutable object) : 数字、字符串、元组、None等
- 相比于可变对象, 不可变对象的访问速度更快

对象一旦被创建, 其id与type便无法再修改, 但可变对象的值却能被修改



■ 变量类型 - type() 函数

Data Type	Value returned by type()	Example
Integer	int	type(5)
Float	float	type(5.5)
Complex	complex	type(5+6j)
Boolean	bool	type(True)
String	str	type("Steve")
List	list	type([1,2,3,4])
Tuple	tuple	type((1,2,3,4))
Set	set	type({1,2,3,4})
Dictionary	dict	type({"name": "Bill", "id": 25})



■ 类型转换介绍

- 隐式类型转换（向上转换） Implicit Type Casting (or Implicit Type Conversion or Upcasting)

```
product_1_Price = 5
```

```
product_2_Price = 5.5
```

```
sum = product_1_Price + product_2_Price
```

```
print(sum)
```




■ 类型转换介绍

— 显式类型转换 Explicit Type Casting (or Explicit Type Conversion)

Datatype	Function used for casting
Integer	int()
Float	float()
Complex	complex()
Boolean	bool()
String	str()
List	list()
Tuple	tuple()
Set	set()



■ Python 中的数字:

- 整数 Integer (int)
- 浮点数 Float (float)
- 复数 Complex (complex)



■ Python 中的二进制、八进制和十六进制：

— 二进制数 Binary Numbers (or base 2 numbers)

- 使用 `bin()` 将 100 转换为二进制数，前缀为 “0b”

— 八进制数 Octal Numbers (or base 8 numbers)

- 使用 `oct()` 将 100 转换为八进制数，前缀为 “0o”

— 十六进制数 Hexadecimal Numbers (or base 16 numbers)

- 使用 `hex()` 将 100 转换为十六进制数，前缀为 “0x”



■ Python 字符串:

- Python字符串是字符的集合
- Python将字符串存储为一个序列，可使用索引位置访问序列中的任何元素
- 可以使用单引号或双引号来创建字符串

- `print('I'm trying to create an error ')`

此字符串会导致错误，因为 `"I' m "` 结束了字符串。要解决这类错误，可使用双引号（`""`）

- `print("I'm trying to fix an error")`



■ Python 字符串 - 转义字符

Escape Sequence	Output
<code>\n</code>	New Line
<code>\\</code>	Backslash
<code>\'</code>	Single Quote
<code>\"</code>	Double Quote
<code>\t</code>	Tab
<code>\b</code>	Backspace
<code>\r</code>	Carriage Return
<code>\v</code>	Vertical Tab



■ Python 字符串:

— 多行字符串 Multiline String

- 可以使用三引号创建多行字符串
- 可以使用单三引号或双三引号，它还将保留制表符和空白

```
greeting = """  
Hello,  
    How are you.
```

```
Thanks for visiting.  
"""
```

```
greeting = '''  
Hello,  
    How are you.
```

```
Thanks for visiting.  
'''
```

■ Python 字符串:

+ - - - +	- - - +	- - - +	- - - +	- - - +	- - - +	- - - +
P	y	t	h	o	n	
0	1	2	3	4	5	
-6	-5	-4	-3	-2	-1	



■ Python 字符串:

- [:]
- + *
- str = 'Hello World!'
 - print (str) # Prints complete string
 - print (str[0]) # Prints first character of the string
 - print (str[2:5]) # Prints characters starting from 3rd to 5th (包头不包尾)
 - print (str[2:]) # Prints string starting from 3rd character
 - print (str * 2) # Prints string two times
 - print (str + "TEST") # Prints concatenated string



■ Python String – Various String Operators

Operator	Name	Description	Example
+	Concatenation	Join the strings	str1 + str2
*	Repetition	Replicate the string	str1 * n
[]	Slice	Returns character at index position	str[i]
[:]	Range Slice	To fetch substring	str[i:j]
in	Membership	Returns true if sub-string is present in string	x in str
not in	Membership	Returns true if sub-string is not present in string	x not in str
r/R	Raw String	Raw string, to prevent escape sequence interpretation	
%	Format	String formatting	



■ Python String Methods

join(seq) 1

len(string) 1

count(str, beg= 0,end=len(string)) 1

capitalize() 1

title() 1

upper() 1

lower() 1

swapcase() 1

translate(table, deletechars="") 1

index(str, beg=0, end=len(string)) 1

rindex(str, beg=0, end=len(string)) 1

find(str, beg=0, end=len(string)) 1

rfind(str, beg=0,end=len(string)) 1

rjust(width,[, fillchar]) 1

center(width, fillchar) 1

ljust(width[, fillchar]) 1

max(str) 1

min(str) 1

replace(old, new [, max]) 1

isalnum() 1

isalpha() 1

isdigit() 1

islower() 1

isnumeric() 1

isspace() 1

istitle() 1

isupper() 1

isdecimal() 1

encode(encoding='UTF-8',errors='strict') 1

bytes.decode(encoding="utf-8",
errors="strict") 1

startswith(substr, beg=0,end=len(string)) 1

endswith(suffix, beg=0, end=len(string)) 1

strip([chars]) 1

rstrip() 1

lstrip() 1

splitlines([keepends]) 1

split(str="", num=string.count(str)) 1

zfill (width) 1

expandtabs(tabsize=8) 1

maketrans() 1



■ 列表 List:

- [:]

- + *

- `list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]`

- `tinylist = [123, 'john']`

- `print (list)` # Prints complete list
 - `print (list[0])` # Prints first element of the list
 - `print (list[1:3])` # Prints elements starting from 2nd till 3rd
 - `print (list[2:])` # Prints elements starting from 3rd element
 - `print (tinylist * 2)` # Prints list two times
 - `print (list + tinylist)` # Prints concatenated lists



■ 列表 List:

```
list = ['physics', 'chemistry', 1997, 2000]  
print (list)
```

```
del list[2]  
print ("After deleting value at index 2 : ", list)
```



■ 列表List Methods

- list.append(obj)
- list.extend(seq)
- list.pop(obj = list[-1])
- list.clear()
- list.remove(obj)
- list.insert(index, obj)
- list.reverse()
- list.sort([func])
- list.count(obj)
- list.index(obj)

— 易错题

```
nums = [1, 2, 2, 2, 3, 4, 2]
```

```
for num in nums:  
    print(num)  
    if num == 2:  
        nums.remove(2)
```

```
print(nums)
```

1

2

2

4

2

```
[1, 3, 4, 2]
```



■ 列表List Methods

- `sort()`
- `reverse()`
- `sorted()`
- `reversed()`

- `lst = [1, 2, 3, 4, 5]`

- `lst.reverse()`

改变lst, 返回None

- `lst[::-1]`

不改变lst, 返回新的排序对象

- 比较两者区别

```
s = 'acbegscvAB'
result = sorted(s, reverse=True)
print(result)
```

```
['v', 's', 'g', 'e', 'c', 'c', 'b', 'a', 'B', 'A']
```

```
sorted(itreable, key=None, reverse=False)
```

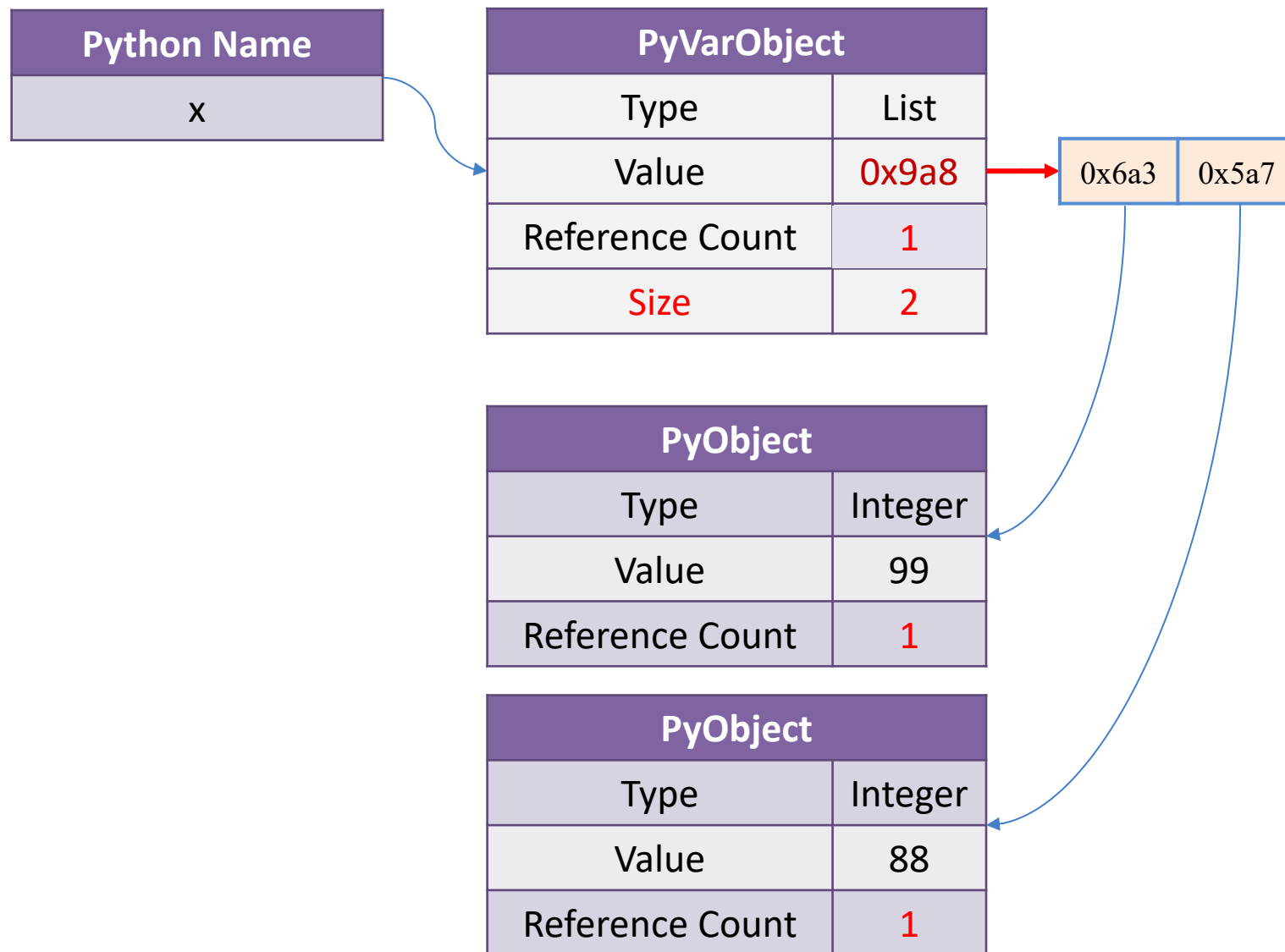
`sorted()` 返回一个排序列表

`reserved()` 返回一个逆序序列的迭代器

■ 变量赋值进阶

— Python赋值

`x = [99, 88]`

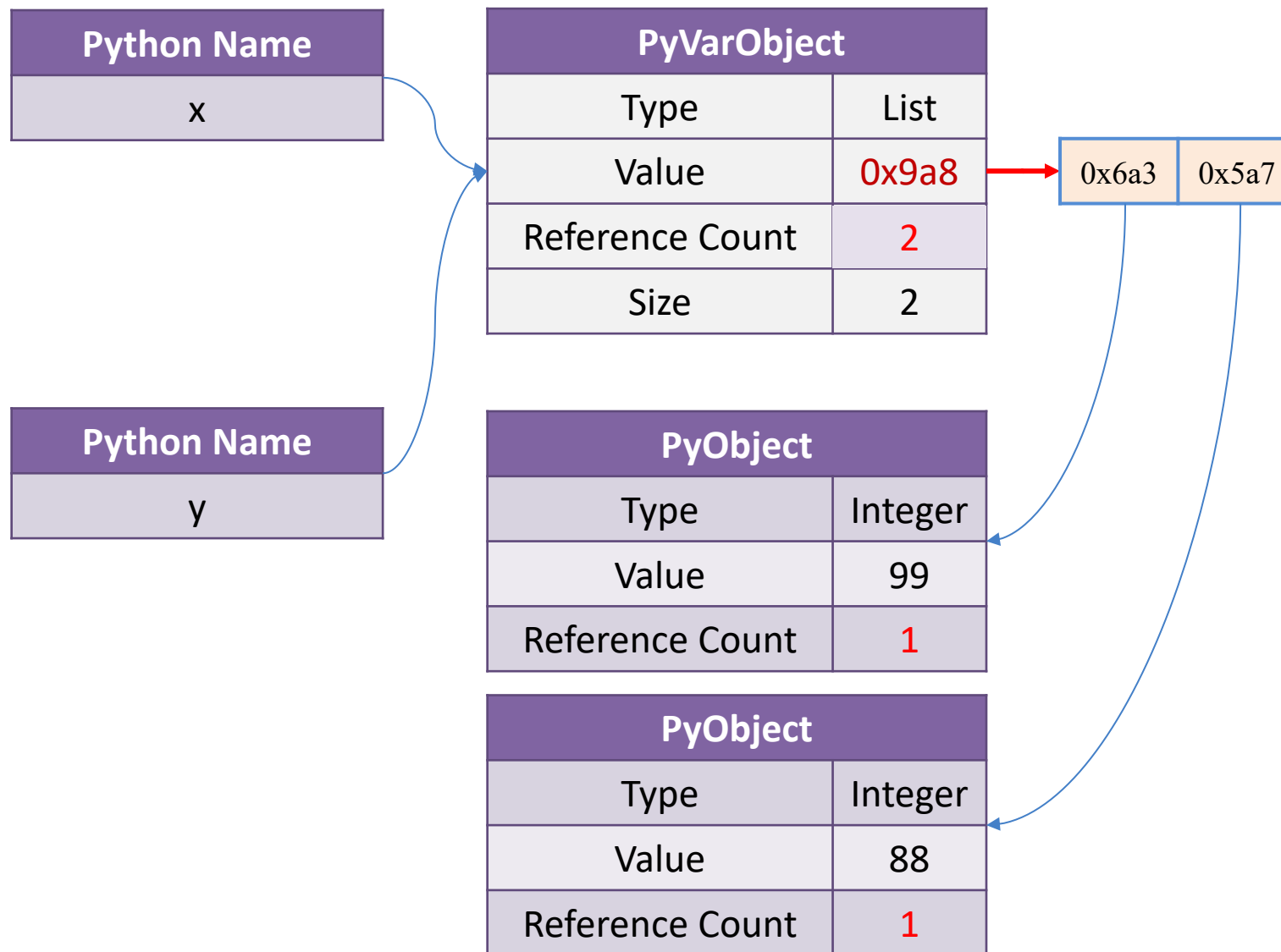


■ 变量赋值进阶

— Python赋值

```
x = [99, 88]
```

```
y = x
```



■ 变量赋值进阶

— Python赋值

```
x = [99, 88]
```

```
y = x
```

```
y.append(7.7)
```

Python Name
x

PyVarObject	
Type	List
Value	0x9a8
Reference Count	2
Size	3

Python Name
y

PyObject	
Type	Integer
Value	99
Reference Count	1

PyObject	
Type	Integer
Value	88
Reference Count	1

0x6a3	0x5a7	0x9f9
-------	-------	-------

PyObject	
Type	Float
Value	7.7
Reference Count	1

■ 变量赋值进阶

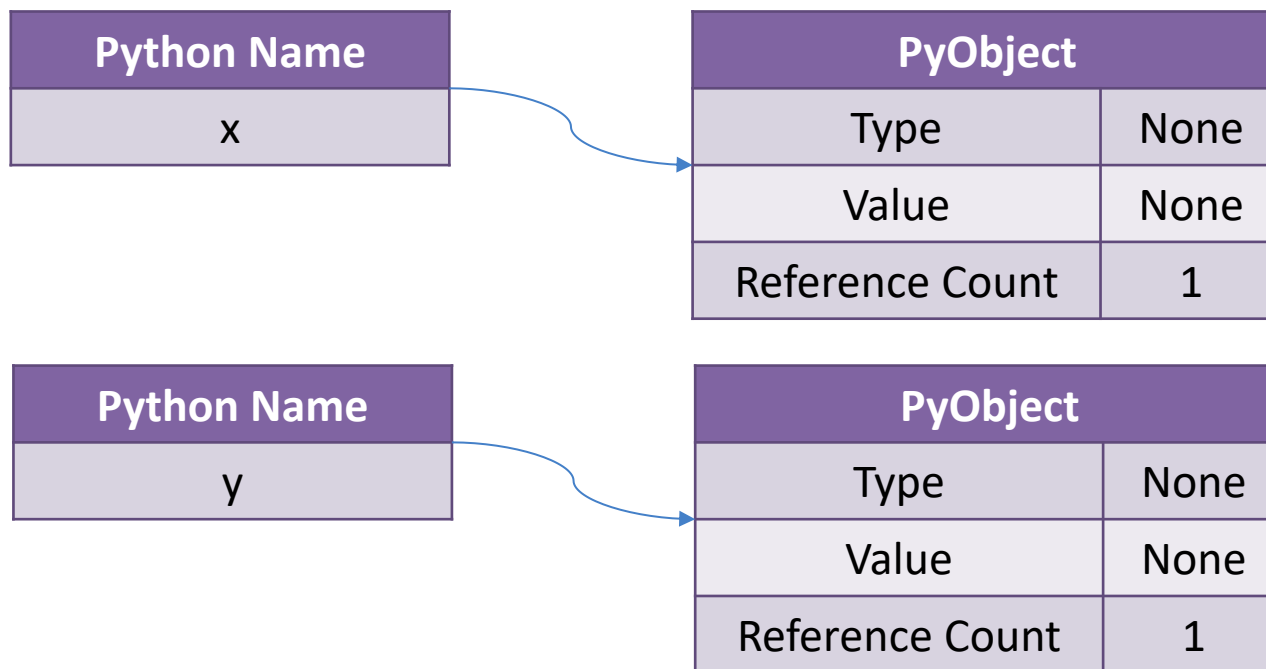
— Python赋值

`x = None`

`y = None`

`x == y`

`x is y`

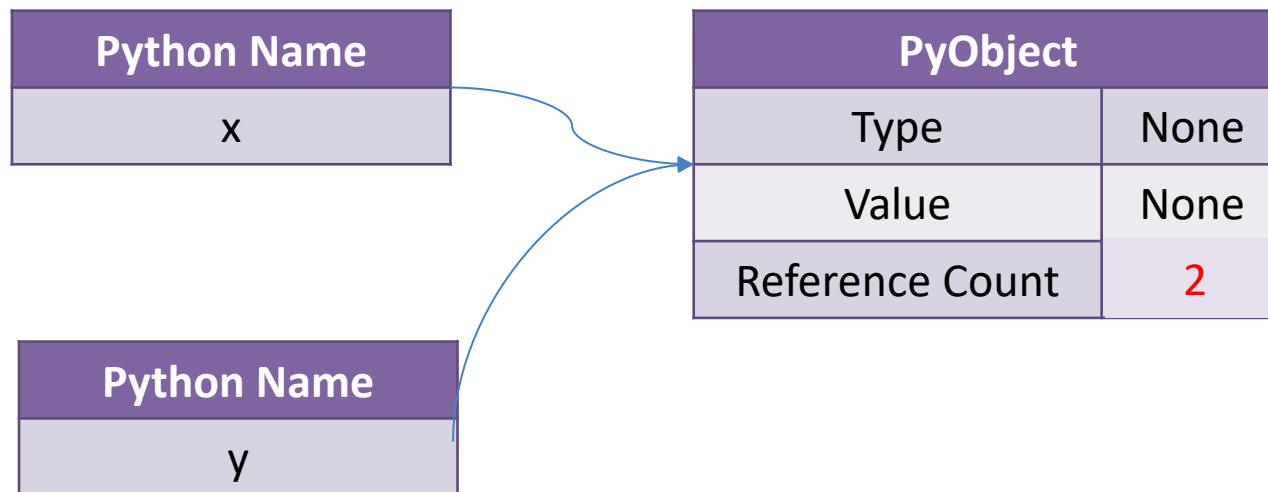


■ 变量赋值进阶

— Python赋值

`x = None`

`y = None`



■ 变量赋值进阶

— Interning in Python (Optimization)

- ✧ None
- ✧ Address of Integer Inside Range (-5 to 256)
- ✧ Strings字符串采用驻留机制
 - 字符串长度为0或1
 - 字符串长度大于1时，且字符串中只包含大小写字母、数字、下划线
 - 字符串只在编译时进行驻留，而非运行时
- ✧ Python提供intern方法强制驻留

```
>>> str1='sten'+ 'waves' # compile_time
>>> str1 is 'stenwaves'
True
```

```
>>> str3='sten'
>>> str4=str3+'waves'
>>> str4 is 'stenwaves' # run_time
False
```

```
>>> import sys
>>> a="a*&&"
>>> b="a*&&"
>>> a is b
False
>>> a=sys.intern(b)
>>> a is b
True
```



■ 元组 tuple

- `tup = ()` 创建一个空元组

- 单元素元组的表示 `(1,)`

- `[:]`

- `+` `*`

- `tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )`

- `tiny_tuple = (123, 'john')`

- `print (tuple)` `# Prints complete tuple`

- `print (tuple[0])` `# Prints first element of the tuple`

- `print (tuple[1:3])` `# Prints elements starting from 2nd till 3rd`

- `print (tuple[2:])` `# Prints elements starting from 3rd element`

- `print (tiny_tuple * 2)` `# Prints tuple two times`

- `print (tuple + tiny_tuple)` `# Prints concatenated tuple`

■ 序列与其他类型的对比

`(1, 2, 3) < (1, 2, 4)`

`[1, 2, 3] < [1, 2, 4]`

`'ABC' < 'C' < 'Pascal' < 'Python'`

`(1, 2, 3, 4) < (1, 2, 4)`

`(1, 2) < (1, 2, -1)`

`(1, 2, 3) == (1.0, 2.0, 3.0)`

`(1, 2, ('aa', 'ab')) < (1, 2, ('abc', 'a'), 4)`

`(1, 2, ('aa', 'ab')) < (1, 2, ('abc'))`



■ 序列与其他类型的对比

- 序列对象通常可以与具有相同序列类型的其他对象进行比较。
- 比较使用词典排序（字符串的词典排序使用 Unicode 代码点编号对单个字符进行排序）：
 - 首先对前两个项目进行比较，如果两者不同，则决定比较的结果；
 - 如果相等，则对下两个项目进行比较，依此类推，直到任一序列耗尽为止。
 - 如果要比较的两个项目本身是同一类型的序列，则会递归进行词性比较。
 - 如果两个序列的所有项目比较结果相等，则认为这两个序列相等。
 - 如果一个序列是另一个序列的初始子序列，则较短的序列是较小（较少）的序列。
- 注意，使用 `<` 或 `>` 比较不同类型的对象是合法的，前提是这些对象具有适当的比较方法。
例如混合数值类型根据其数值进行比较，因此等于 `0 == 0.0`，结果是True。

■ 字典 Dictionary

- `dict = {}`
- `dict['one'] = "This is one"`
- `dict[2] = "This is two"`

- `tinydict = {'name': 'john', 'code': 6734, 'dept': 'sales'}`
 - `print (dict['one'])` # Prints value for 'one' key
 - `print (dict[2])` # Prints value for 2 key
 - `print (tinydict)` # Prints complete dictionary
 - `print (tinydict.keys())` # Prints all the keys
 - `print (tinydict.values())` # Prints all the values

■ 字典 Dictionary Method & Description

- **dict.fromkeys()**

Create a new dictionary with keys from seq and values set to value.

- **dict.update(dict2)**

Adds dictionary dict2's key-values pairs to dict

- **dict.copy()**

Returns a shallow copy of dictionary dict

- **dict.setdefault(key, default = None)**

Similar to get(), but will set dict[key] = default if key is not already in dict

- ~~**dict.has_key(key)**~~

Removed, use the in operation instead **Python3已移除该方法，改用 in 或 not in**

- **dict.clear()**

Removes all elements of dictionary dict

- **del / dict.pop() / dict.popitem()**

- **dict.get(key, default=None)**

For key key, returns value or default if key not in dictionary

- **dict.items()**

Returns a list of dict's (key, value) tuple pairs

- **dict.keys()**

Returns list of dictionary dict's keys

- **dict.values()**

Returns list of dictionary dict's values



```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
```

- `del dict['Name']` # remove entry with key 'Name'
- `del dict` # delete entire dictionary
- `dict.clear()` # remove all entries in dict



字典dict特点:

- 查找和插入的速度极快, 不会随着key的增加而变慢;
- 需要占用大量的内存, 内存浪费多。

列表list特点:

- 查找和插入的时间随着元素的增加而增加;
- 占用空间小, 浪费内存很少。

字典dict是用空间来换取时间的一种方法。



■ Python 集合 (Set) :

- 集合是无序的唯一元素的集合。
- 我们使用大括号"{}"来定义集合。默认情况下, 这些大括号用于字典, 但如果我们在这些大括号"{}"中使用逗号分隔元素列表, 则会将其视为集合。
- 我们也可以使用 `set()` 函数创建集合。
- 集合中的项目没有索引位置。
- 集合是可变的。



■ Python Set Methods:

Method	Description
add()	To add a new item to the set.
clear()	This method will delete all the elements from the set.
copy()	Creates a new copy of the set.
difference()	Returns the item that is unique to the first set.
discard()	Removes the item from the set.
intersection()	Returns the set of items that are common to both sets.
issubset()	Checks if set 'a' contains all the items of set 'b'.
issuperset()	Checks if set 'b' contains all the items of set 'a'.
pop()	To remove an item from the set.
remove()	Removes the specified item from the set.
union()	It returns the combined set of unique items of the sets.
update()	It updates set 'a' with unique values of set 'b'.



■ Python bytes类型

- **bytes 是 Python 3.x 新增的类型，在 Python 2.x 中是没有的。**
- **bytes 以字节序列的形式（二进制形式）来存储数据，至于这些数据到底表示什么内容（字符串、数字、图片、音频等），完全由程序的解析方式决定。如果采用合适的字符编码方式（字符集），字节串可以恢复成字符串；反之亦然，字符串也可以转换成字节串。**

```
1  #通过构造函数创建空 bytes
2  b1 = bytes()
3  #通过空字符串创建空 bytes
4  b2 = b''
5
6  #通过b前缀将字符串转换成 bytes
7  b3 = b'http://c.biancheng.net/python/'
8  print("b3: ", b3)
9  print(b3[3])
10 print(b3[7:22])
11
12 #为 bytes() 方法指定字符集
13 b4 = bytes('C语言中文网8岁了', encoding='UTF-8')
14 print("b4: ", b4)
15
16 #通过 encode() 方法将字符串转换成 bytes
17 b5 = "C语言中文网8岁了".encode('UTF-8')
18 print("b5: ", b5)
```

运行结果:

```
b3: b'http://c.biancheng.net/python/'
112
b'c.biancheng.net'
b4:
b'C\xe8\xaf\xad\xe8\xa8\x80\xe4\xb8\xad\xe6\x96
\x87\xe7\xbd\x918\xe5\xb2\x81\xe4\xba\x86'
b5:
b'C\xe8\xaf\xad\xe8\xa8\x80\xe4\xb8\xad\xe6\x96
\x87\xe7\xbd\x918\xe5\xb2\x81\xe4\xba\x86'
```



```
>>> b'ABC'.decode('ascii')  
'ABC'
```

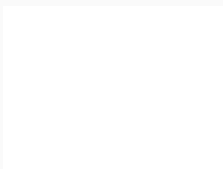
```
>>> b'\xe4\xb8\xad\xe6\x96\x87'.decode('utf-8')  
'中文'
```



```
>>> len('ABC')
```

```
3
```

```
>>> len('中文')
```





```
>>> len(b'ABC')
```

```
>>> len(b' \xe4\xb8\xad\xe6\x96\x87')
```

```
>>> len('中文'.encode('utf-8'))
```



■ 格式化输出 Formatted Output

- Formatting with % Operator
- Formatting with `format()` string method
- Formatting with string literals, called `f-strings`
- Formatting with String Template Class

■ 格式化输出 Formatted Output

— Formatting with % Operator

```
x = 'looked'  
print("Misha %s and %s around." % ('walked', x))
```

Misha walked and looked around.

```
print('The value of pi is: %5.4f' % 3.141592)
```

The value of pi is: 3.1416

```
print('Floating point numbers: %1.0f' % 13.144)
```

Floating point numbers: 13

■ 格式化输出 Formatted Output

- Formatting with `format()` string method

Syntax: 'String here {} then also {}'.format('something1','something2')

```
print('{2} {1} {0}'.format('directions', 'the', 'Read'))
```

Read the directions

```
print('a: {a}, b: {b}, c: {c}'.format(a=1,  
                                       b='Two',  
                                       c=12.3))
```



■ 格式化输出 Formatted Output

— Formatting with format() string method

Syntax: {[index]:[width][.precision][type]}

The type can be used with format codes:

- 'd' for integers
 - 'f' for floating-point numbers
 - 'b' for binary numbers
 - 'o' for octal numbers
 - 'x' for octal hexadecimal numbers
 - 's' for string
 - 'e' for floating-point in an exponent format
- The value of pi is: 3.14159

```
print('The value of pi is: {0:1.5f}'.format(3.141592))
```

■ 格式化输出 Formatted Output

- Formatting with string literals, called **f-strings**

```
name = 'Ele'
print(f"My name is {name}.")

print(f"He said his age is {(lambda x: x * 2)(3)}")
```

Float precision in the f-String method:

Syntax: {value:{width}.{precision}}

```
num = 3.14159
print(f"The value of pi is: {num:{1}.{5}}")
```




■ 格式化输出 Formatted Output

— Formatting with String Template Class

In the String module, Template Class allows us to create simplified syntax for output specification. The format uses placeholder names formed by \$ with valid Python identifiers (alphanumeric characters and underscores). Surrounding the placeholder with braces allows it to be followed by more alphanumeric letters with no intervening spaces. Writing \$\$ creates a single escaped \$:

■ 格式化输出 Formatted Output

- Formatting with String Template Class

```
from string import Template  
  
n1 = 'Hello'  
n2 = 'GeeksforGeeks'  
n = Template('$n3 ! This is $n4.')  
print(n.substitute(n3=n1, n4=n2))
```

Hello ! This is GeeksforGeeks.



Python Types of Operator(Python 运算类型)

- Arithmetic Operators
- Comparison (Relational) Operators
- Assignment Operators
- Logical Operators
- Bitwise Operators
- Membership Operators
- Identity Operators

Python Arithmetic Operators

a = 10 b = 21

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 31$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -11$
* Multiplication	Multiplies values on either side of the operator	$a * b = 210$
/ Division	Divides left hand operand by right hand operand	$b / a = 2.1$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 1$
** Exponent	Performs exponential (power) calculation on operators	$a ** b$ =10 to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	$9 // 2 = 4$ $9.0 // 2.0 = 4.0$, $-11 // 3 = -4$, $-11.0 // 3 = -4.0$



Python Comparison Operators

```
a = 3  
print(2 < a < 7)
```

Operator	Description	Example
==	If the values of two operands are equal, then the condition becomes true.	(a == b) is not true.
!=	If values of two operands are not equal, then condition becomes true.	(a != b) is true.
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	(a > b) is not true.
<	If the value of left operand is less than the value of right operand, then condition becomes true.	(a < b) is true.
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	(a >= b) is not true.
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	(a <= b) is true.



Python Assignment Operators

Operator	Description	Example
=	Assigns values from right side operands to left side operand	<code>c = a + b</code> assigns value of <code>a + b</code> into <code>c</code>
<code>+=</code> Add AND	It adds right operand to the left operand and assign the result to left operand	<code>c += a</code> is equivalent to <code>c = c + a</code>
<code>-=</code> Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	<code>c -= a</code> is equivalent to <code>c = c - a</code>
<code>*=</code> Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	<code>c *= a</code> is equivalent to <code>c = c * a</code>
<code>/=</code> Divide AND	It divides left operand with the right operand and assign the result to left operand	<code>c /= a</code> is equivalent to <code>c = c / a</code>
<code>%=</code> Modulus AND	It takes modulus using two operands and assign the result to left operand	<code>c %= a</code> is equivalent to <code>c = c % a</code>
<code>**=</code> Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	<code>c **= a</code> is equivalent to <code>c = c ** a</code>
<code>//=</code> Floor Division	It performs floor division on operators and assign value to the left operand	<code>c //= a</code> is equivalent to <code>c = c // a</code>

Python Bitwise Operators

Operator	Description	Example
& Binary AND	Operator copies a bit, to the result, if it exists in both operands	(a & b) (means 0000 1100)
Binary OR	It copies a bit, if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR 异或	It copies the bit, if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operand's value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operand's value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)



Python Bitwise Operators

a = 0011 1100

b = 0000 1101

a&b = 0000 1100

a|b = 0011 1101

a^b = 0011 0001

~a = 1100 0011

Python Logical Operators

逻辑运算符	含义	基本格式	说明
and	逻辑与运算，等价于数学中的“且”	a and b	当 a 和 b 两个表达式都为真时，a and b 的结果才为真，否则为假
or	逻辑或运算，等价于数学中的“或”	a or b	当 a 和 b 两个表达式都为假时，a or b 的结果才是假，否则为真。
not	逻辑非运算，等价于数学中的“非”	not a	如果 a 为真，那么 not a 的结果为假；如果 a 为假，那么 not a 的结果为真。相当于对 a 取反。



Python Membership Operators

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	<code>x in y</code> , here <code>in</code> results in a 1 if <code>x</code> is a member of sequence <code>y</code> .
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	<code>x not in y</code> , here <code>not in</code> results in a 1 if <code>x</code> is not a member of sequence <code>y</code> .



Python Identity Operators

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).

```
1  a = 20
2  b = 20
3  print ('Line 1','a=',a,':',id(a), 'b=',b,':',id(b))
4
5  if ( a is b ):
6      | print ("Line 2 - a and b have same identity")
7  else:
8      | print ("Line 2 - a and b do not have same identity")
9
10 if ( id(a) == id(b) ):
11     | print ("Line 3 - a and b have same identity")
12 else:
13     | print ("Line 3 - a and b do not have same identity")
14
15 b = 30
16 print ('Line 4','a=',a,':',id(a), 'b=',b,':',id(b))
17
18 if ( a is not b ):
19     | print ("Line 5 - a and b do not have same identity")
20 else:
21     | print ("Line 5 - a and b have same identity")
```




■ 条件判断

```
age = 20
if age >= 18:
    print('your age is', age)
    print('adult')
```



■ 条件判断

```
age = 3
if age >= 18:
    print('your age is', age)
    print('adult')
else:
    print('your age is', age)
    print('teenager')
```



■ if判断的效率问题

```
if 90 <= score <= 100:  
    print("优秀")  
  
if 80 <= score < 90:  
    print("良好")  
  
if 60 <= score < 80:  
    print("及格")  
  
if 0 <= score < 60:  
    print("不及格")
```

```
if 90 <= score <= 100:  
    print("优秀")  
else:  
    if 80 <= score < 90:  
        print("良好")  
    else:  
        if 60 <= score < 80:  
            print("及格")  
        else:  
            if 0 <= score < 60:  
                print("不及格")
```



■ 条件判断

```
age = 3
if age >= 18:
    print('adult')
elif age >= 6:
    print('teenager')
else:
    print('kid')
```



■ 条件判断

```
if <条件判断1>:  
    <执行1>  
elif <条件判断2>:  
    <执行2>  
elif <条件判断3>:  
    <执行3>  
else:  
    <执行4>
```



■ 条件判断易错情形

```
score = 88
```

```
if score < 60:  
    print("E")  
elif score >= 60:  
    print("D")  
elif score >= 70:  
    print("C")  
elif score >= 80:  
    print("B")  
elif score >= 90:  
    print("A")
```




■ switch

```
1  grade = 88
2
3  if grade >= 90:
4      print('A')
5  elif grade >= 80:
6      print('B')
7  elif grade >= 70:
8      print('C')
9  else:
10     print('None')
```



■ switch

```
1   day = 6
2   ✓ switcher = {
3       0: 'Sunday',
4       1: 'Monday',
5       2: 'Tuesday'
6   }
7
8   day_name = switcher.get(day, 'Unknown')
9   print(day_name)
```

■ switch

```
11  day = 0
12
13
14  # 用字典代替switch
15  def get_sunday():
16      return 'Sunday'
17
18
19  def get_monday():
20      return 'monday'
21
22
23  def get_tuesday():
24      return 'tuesday'
25
26
27  def get_default():
28      return 'Unkown'
29
30
31  switcher = {
32      0: get_sunday,
33      1: get_monday,
34      2: get_tuesday
35  }
36
37  day_name = switcher.get(day, get_default())
38  print(day_name)
```

■ match 分支结构

```
match subject:
    case <pattern_1>:
        <action_1>
    case <pattern_2>:
        <action_2>
    case <pattern_3>:
        <action_3>
    case _:
        <action_wildcard>
```

A match statement takes an expression and compares its value to successive patterns given as one or more case blocks. This is superficially similar to a **switch statement** in C, Java or JavaScript (and many other languages), but it's **more similar** to pattern matching in languages like **Rust or Haskell**.

Only the first pattern that matches gets executed and it can also extract components (sequence elements or object attributes) from the value into variables.

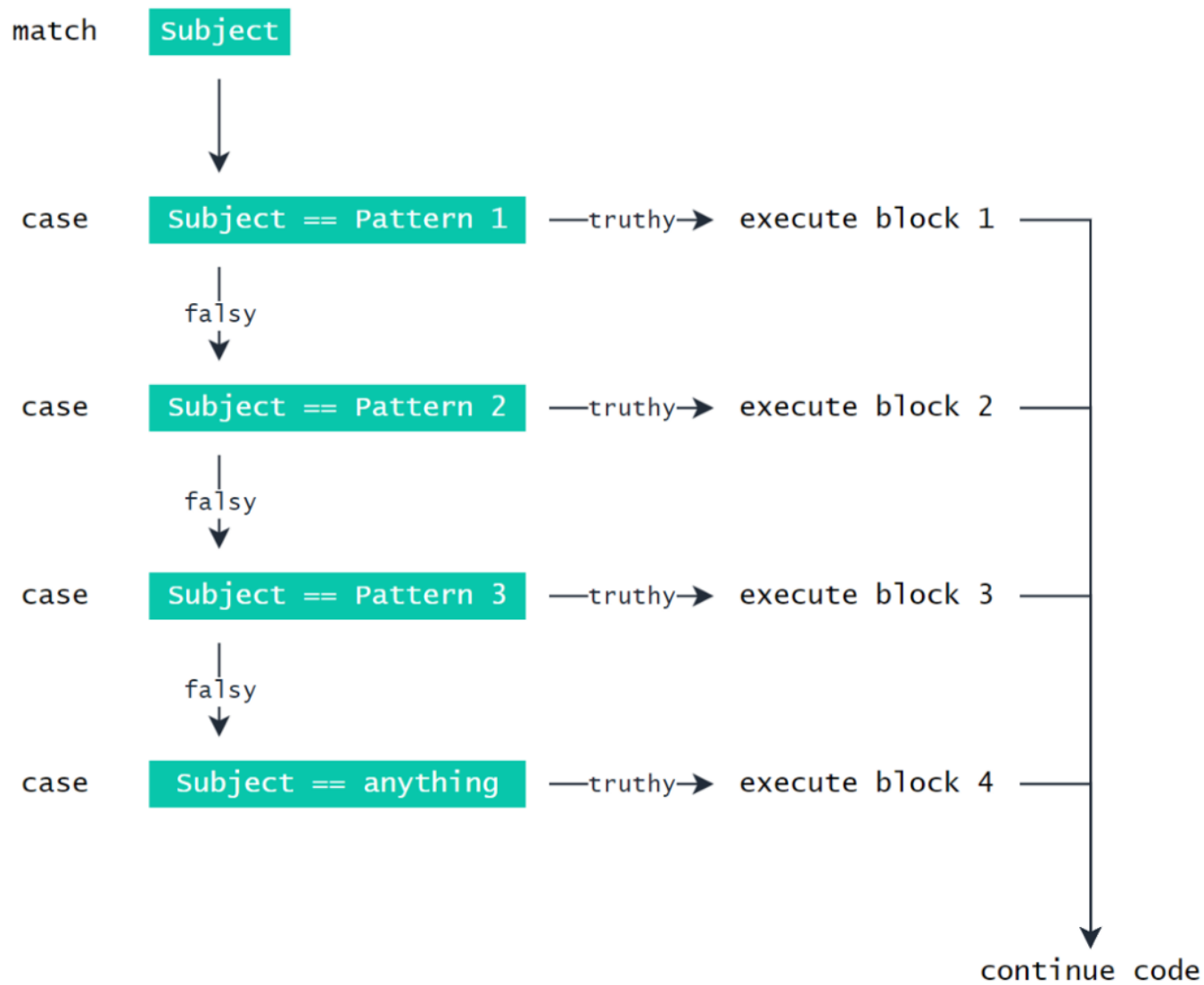
The “variable name” **_** acts as a wildcard and never fails to match. **case _:** 类似于C和Java中的**default:**，永远不会匹配失败，即当其它**case**都无法匹配时，匹配此**case**。



■ match 分支结构

```
def http_error(status):  
    match status:  
        case 400:  
            return "Bad request"  
        case 404:  
            return "Not found"  
        case 418:  
            return "I'm a teapot"  
        case _:  
            return "Something's wrong with the internet"
```

match分支结构



■ match分支结构

```
1  http_code = "418"
2  match http_code:
3      case "200":
4          print("OK")
5          do_something_good()
6      case "404":
7          print("Not Found")
8          do_something_bad()
9      case "418":
10         print("I'm a teapot")
11         make_coffee()
12     case _:
13         print("Code not found")
```

■ if替代switch

```
16 http_code = "418"
17 if http_code == "418":
18     print("OK")
19     do_something_good()
20 elif http_code == "404":
21     print("Not Found")
22     do_something_bad()
23 elif http_code == "418":
24     print("I'm a teapot")
25     make_coffee()
26 else:
27     print("Code not found")
```



■ 循环loop Type & Description

- while loop
- for loop
- nested loops



■ while loop

```
count = 0
while (count < 9):
    print ('The count is:', count)
    count = count + 1

print ("Good bye!")
```



```
count = 0
while count < 5:
    print(count, "is less than 5")
    count += 1
```

else:

```
    print(count, "is not less than 5")
```

```
0 is less than 5
1 is less than 5
2 is less than 5
3 is less than 5
4 is less than 5
5 is not less than 5
```

```
0 is less than 5
1 is less than 5
2 is less than 5
3 is less than 5
```

```
count = 0
while count < 5:
    print(count, "is less than 5")
    if count == 3:
        break
    count += 1
```

else:

```
    print(count, "is not less than 5")
```



■ for loop

```
for x in range(1, 5):  
    print(x, end="*")
```

1*2*3*4*

```
words = ['cat', 'window', 'defenestrate']  
for w in words:  
    print(w, len(w))
```

```
cat 3  
window 6  
defenestrate 12
```




■ for loop

```
for n in range(2, 10):  
    for x in range(2, n):  
        if n % x == 0:  
            print(n, 'equals', x, '*', n // x)  
            break  
        else:  
            # loop fell through without finding a factor  
            print(n, 'is a prime number')
```

```
2 is a prime number  
3 is a prime number  
4 equals 2 * 2  
5 is a prime number  
6 equals 2 * 3  
7 is a prime number  
8 equals 2 * 4  
9 equals 3 * 3
```

When used with a loop, the else clause has more in common with the else clause of a **try** statement than it does with that of if statements: a try statement's else clause runs when no exception occurs, and a loop's else clause runs when no break occurs.

■ for loop案例

去掉users中值为'inactive'的用户(键值对)

```
users = {'Hans': 'active', 'Éléonore': 'inactive', '景太郎': 'active'}
```

```
for user, status in users.copy().items():  
    if status == 'inactive':  
        del users[user]
```

```
active_users = {}  
for user, status in users.items():  
    if status == 'active':  
        active_users[user] = status
```



- **break**
- **continue**
- **pass**

■ 内置函数示例

```
x = abs(100)
```

```
y = abs(-20)
```

```
print(x, y)
```

```
print('max(1, 2, 3) =', max(1, 2, 3))
```

```
print('min(1, 2, 3) =', min(1, 2, 3))
```

```
print('sum([1, 2, 3]) =', sum([1, 2, 3]))
```

Built-in Functions

A

```
abs()
aiter()
all()
any()
anext()
ascii()
```

B

```
bin()
bool()
breakpoint()
bytearray()
bytes()
```

C

```
callable()
chr()
classmethod()
compile()
complex()
```

D

```
delattr()
dict()
dir()
divmod()
```

E

```
enumerate()
eval()
exec()
```

F

```
filter()
float()
format()
frozenset()
```

G

```
getattr()
globals()
```

H

```
hasattr()
hash()
help()
hex()
```

I

```
id()
input()
int()
isinstance()
issubclass()
iter()
```

L

```
len()
list()
locals()
```

M

```
map()
max()
memoryview()
min()
```

N

```
next()
```

O

```
object()
oct()
open()
ord()
```

P

```
pow()
print()
property()
```

R

```
range()
repr()
reversed()
round()
```

S

```
set()
setattr()
slice()
sorted()
staticmethod()
str()
sum()
super()
```

T

```
tuple()
type()
```

V

```
vars()
```

Z

```
zip()
```

```
__import__()
```



■ 自定义函数

```
def my_abs(x):  
    if x >= 0:  
        return x  
    else:  
        return -x
```



■ Python函数要点:

- 定义函数时，需要确定函数名和参数个数；
- 如有必要，可以先对参数的数据类型做检查；
- 函数体内部可以用return随时返回函数结果；
- 函数执行完毕也没有return语句时，自动return None；
- 函数可以同时返回多个值，但其实就是一个tuple。

■ 位置参数 positional parameters

```
def power(x, n):  
    s = 1  
    while n > 0:  
        n = n - 1  
        s = s * x  
    return s
```




■ 默认参数

```
def enroll(name, gender, age=6, city='Beijing'):  
    print('name:', name)  
    print('gender:', gender)  
    print('age:', age)  
    print('city:', city)
```

默认参数必须在最右端，才能被解释器正确识别，否则会产生歧义

■ 易错案例

```
def add_end(lst=[]):  
    lst.append('END')  
    return lst
```

```
def add_end(lst=[]):  
    lst.append('END')  
    return lst
```

```
print(add_end())  
print(add_end())
```

```
['END']
```

```
['END', 'END']
```

默认参数值在函数被定义时
已创建，而非在程序运行时

```
def func(i=1):  
    print(id(i))  
    i += 1  
    print(id(i))
```

```
print(func.__defaults__)
```

```
(1,)
```



■ 正确的写法

```
def add_end(lst=None):
```

```
    if lst is None:
```

```
        lst = []
```

```
        lst.append('END' )
```

```
    return lst
```



■ 易错案例

```
def buggy(arg, result=[]):  
    result.append(arg)  
    print(result)
```

```
print('-----1-----')  
buggy('a')  
print('-----2-----')  
buggy('b') # 期望得到 ['b']
```

```
-----1-----  
['a']  
-----2-----  
['a', 'b']
```

■ 易错案例

```
def buggy(arg, result=[]):  
    print(id(result))  
    result.append(arg)  
    print(result)
```

```
print('-----1-----')  
buggy('a')  
print('-----2-----')  
buggy('b') # expect ['b']
```

只要函数调用时没有传递新的列表来覆盖默认参数列表，函数会使用定义时的列表，并且操作依次叠加。

```
-----1-----  
55248776  
['a']  
-----2-----  
55248776  
['a', 'b']
```




■ 易错案例

```
def buggy(arg, result=[]):  
    print(id(result))  
    result.append(arg)  
    print(result)
```

```
print('-----1-----')  
buggy('a')  
print('-----2-----')  
buggy('b', []) # 传递了新的列表
```

```
-----1-----  
46073736  
['a']  
-----2-----  
46086784  
['b']
```



■ 正确的写法

```
def works(arg):  
    result = []  
    result.append(arg)  
    print(result)
```

```
works('a')  
works('b')
```

```
['a']  
['b']
```



■ 函数参数传递的本质:

- 对“可变对象” (mutable object)进行“写操作”，直接作用于原对象本身
 - 对“不可变对象” (immutable object) “写操作”，会产生一个新的“对象空间”
-
- 可变对象：字典、列表、集合、自定义对象（依赖用户定义的对象特征）等
 - 不可变对象：数字、字符串、元组、None等

给定一组数字a, b, c.....,

请计算 $a^2 + b^2 + c^2 + \dots$

```
def calc(numbers):  
    sum = 0  
    for n in numbers:  
        sum = sum + n * n  
    return sum
```

■ 调用的时候, 需要先组装
出一个list或tuple:
calc([1, 2, 3])

■ 可变参数

给定一组数字 $a, b, c, \dots$,

请计算 $a^2 + b^2 + c^2 + \dots$

```
def calc(*numbers):  
    sum = 0  
    for n in numbers:  
        sum = sum + n * n  
    return sum
```



```
def calc(*numbers):  
    sum = 0  
    for n in numbers:  
        sum = sum + n * n  
    return sum  
  
nums = [1, 2, 3]  
  
calc(nums[0], nums[1], nums[2])  
calc(*nums)
```



■ 关键字参数

```
def person(name, age, **kw):  
    print('name:', name, 'age:', age, 'other:', kw)  
  
person('Michael', 30)  
name: Michael age: 30 other: {}  
  
person('Bob', 35, city='Beijing')  
name: Bob age: 35 other: {'city': 'Beijing'}  
  
person('Adam', 45, gender='M', job='Engineer')  
name: Adam age: 45 other: {'gender': 'M', 'job': 'Engineer'}  
  
extra = {'city': 'Beijing', 'job': 'Engineer'}  
person('Jack', 24, **extra)
```




```
def person(name, age, **kw):
```

```
    if 'city' in kw:
```

```
        # 有city参数
```

```
        pass
```

```
    if 'job' in kw:
```

```
        # 有job参数
```

```
        pass
```

```
    print('name:', name, 'age:', age, 'other:', kw)
```

```
person('Jack', 24, city='Beijing', addr='Chaoyang', zipcode=123456)
```

■ 参数调用

```
def person(name, age, *, city, job):  
    print(name, age, city, job)
```

```
person('Jack', 24, 'Beijing', 'Engineer')
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
TypeError: person() takes 2 positional arguments but 4 were given
```

如果没有传入参数名，调用将报错



■ 命名关键字参数

```
def person_info(name, age, *, gender, city):  
    print(name, age, gender, city)
```

```
person_info('Steve', 22, gender='male', city='shanghai')
```

```
Steve 22 male shanghai
```



■ 参数组合

```
def f1(a, b, c=0, *args, **kw):  
    print('a =', a, 'b =', b, 'c =', c, 'args =', args, 'kw =', kw)
```

```
def f2(a, b, c=0, *, d, **kw):  
    print('a =', a, 'b =', b, 'c =', c, 'd =', d, 'kw =', kw)
```

问题：f2如何调用？



- **参数组合** 可读性是衡量代码优劣的一个重要指标，请尽量**不要使用**多种参数的组合形式

```
>>> f1(1, 2)
```

```
a = 1 b = 2 c = 0 args = () kw = {}
```

```
>>> f1(1, 2, c=3)
```

```
a = 1 b = 2 c = 3 args = () kw = {}
```

```
>>> f1(1, 2, 3, 'a', 'b')
```

```
a = 1 b = 2 c = 3 args = ('a', 'b') kw = {}
```

```
>>> f1(1, 2, 3, 'a', 'b', x=99)
```

```
a = 1 b = 2 c = 3 args = ('a', 'b') kw = {'x': 99}
```

```
>>> f2(1, 2, d=99, ext=None)
```

```
a = 1 b = 2 c = 0 d = 99 kw = {'ext': None}
```



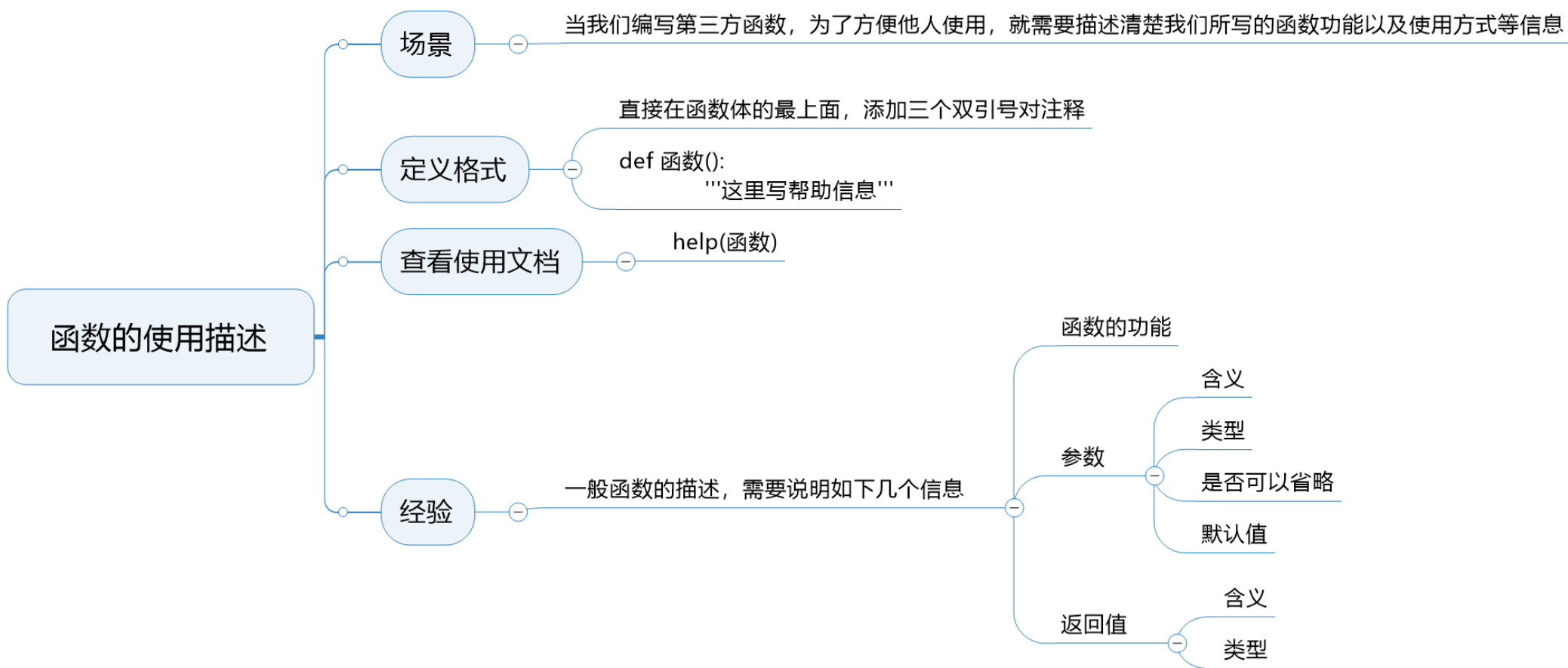
■ 参数组合

```
>>> args = (1, 2, 3, 4)
>>> kw = {'d': 99, 'x': '#'}
>>> f1(*args, **kw)
a = 1 b = 2 c = 3 args = (4,) kw = {'d': 99, 'x': '#'}
>>> args = (1, 2, 3)
>>> kw = {'d': 88, 'x': '#'}
>>> f2(*args, **kw)
a = 1 b = 2 c = 3 d = 88 kw = {'x': '#'}
```

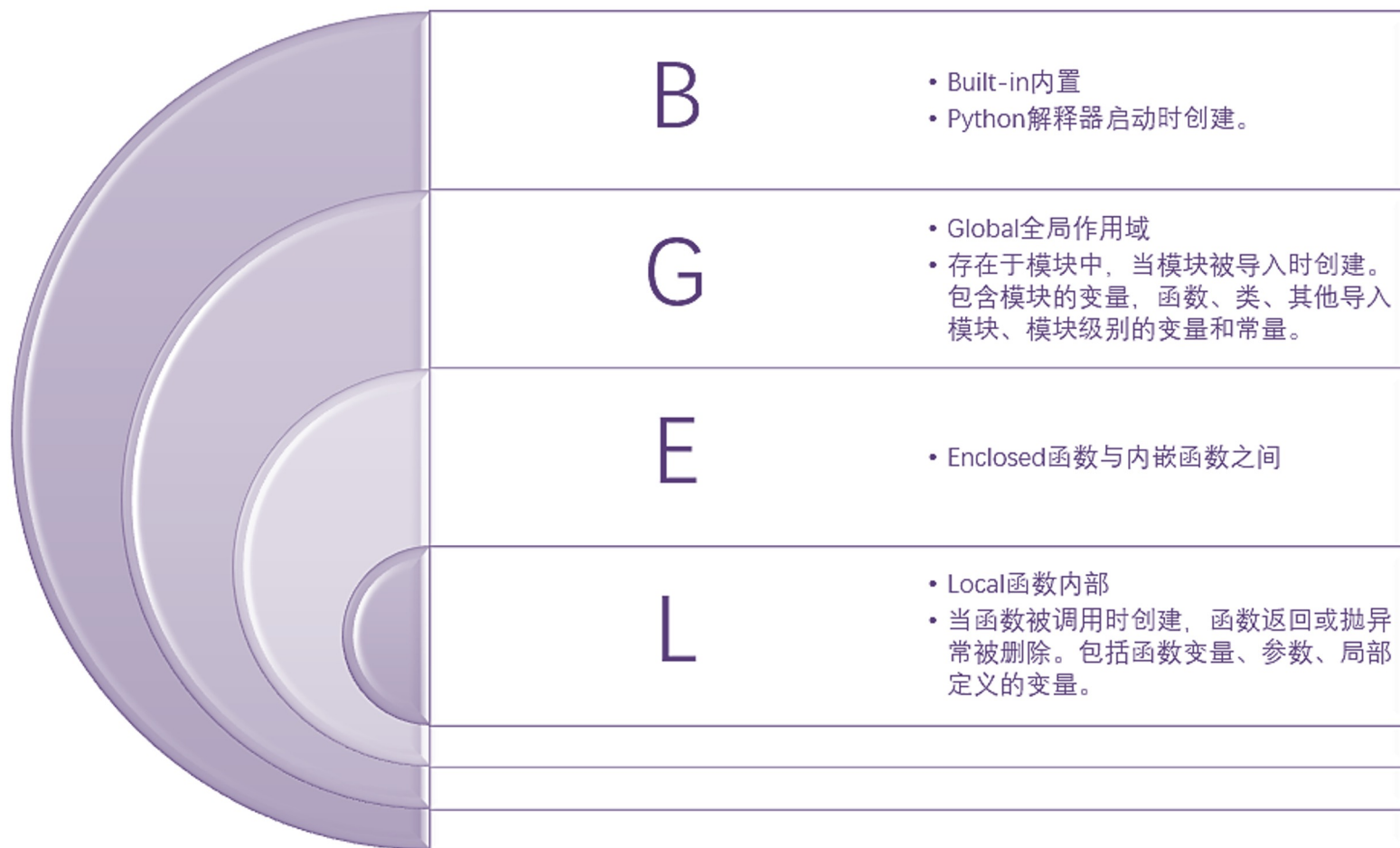


- Python函数具有非常灵活的参数形态，既可以实现简单的调用，又可以传入非常复杂的参数
- 默认参数一定要用不可变对象，如果是可变对象，程序运行时会有逻辑错误
- 要注意定义可变参数和关键字参数的语法：
 - *args是可变参数，args接收的是一个tuple
 - **kw是关键字参数，kw接收的是一个dict
- 调用函数时，传入可变参数和关键字参数的语法：
 - 可变参数既可以直接传入：func(1, 2, 3)，也可以先组装成list或tuple，再通过*args传入：func(*(1, 2, 3))
 - 关键字参数既可以直接传入：func(a=1, b=2)，也可以先组装dict，再通过**kw传入：func(**{'a': 1, 'b': 2})
- 使用*args和**kw是Python的习惯写法，当然也可以用其它参数名，推荐使用习惯写法
- 命名关键字参数，是为了限制调用者可以传入的参数名，同时可以提供默认值
- 定义命名关键字参数在没有可变参数的情况下，不要忘了写分隔符*，否则定义的将是位置参数

函数说明文档



■ Python作用域



■ Python作用域

```
# -----同作用域内-----  
name = "MING"  
print(name)
```

```
# -----不同作用域内-----  
name = "MING"  
def main():  
    print(name)
```



■ Python作用域

变量/函数的查找顺序：

L → E → G → B

会影响 变量/函数 作用范围的有

- 函数：def 或 lambda
- 类：class
- 关键字：global nonlocal
- 文件：*.py
- 推导式：[], {}, ()等，仅限Python3.x中，Python2.x会出现变量泄露



■ Python作用域

```
print(name)  
name = "MING"
```

UnboundLocalError: local variable 'name' referenced before assignment

引用在前，赋值在后（同一作用域内）

■ Python作用域

```
# L -> E -> G -> B
```

```
# 从左到右，由低层到高层
```

```
def main():
```

```
    name = "MING"
```

```
print(name)
```

```
# NameError: name 'name' is not defined
```

赋值在底层，引用在高层



■ globals()与locals ()

```
# 全局变量
py_name = "Python语言程序设计"
py_url = "http://www.binhui.wang"
```

```
def text():
    # 局部变量
    inn_name = "Python作用域"
    inn_url = "http://lab.binhui.wang"
    print("函数内部的 locals:")
    print(locals())
```

```
text()
print("函数外部的 locals:")
print(locals())
print("函数全局的 globals:")
print(globals())
```

函数内部的 locals:

```
{'inn_url': 'http://lab.binhui.wang', 'inn_name': 'Python作用域'}
```

函数外部的 locals:

```
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <frozen_importlib_external.  
SourceFileLoader object at 0x03840890>, '__spec__': None, '__annotations__': {}, '__builtins__': <module  
'builtins' (built-in)>, '__file__': 'C:/Users/B.H WANG/PycharmProjects/untitled/legb.py', '__cached__':  
None, 'py_name': 'Python语言程序设计', 'py_url': 'http://www.binhui.wang', 'text': <function text at 0x03A49108>}
```

函数全局的 globals:

```
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <frozen_importlib_external.  
SourceFileLoader object at 0x03840890>, '__spec__': None, '__annotations__': {}, '__builtins__': <module  
'builtins' (built-in)>, '__file__': 'C:/Users/B.H WANG/PycharmProjects/untitled/legb.py', '__cached__':  
None, 'py_name': 'Python语言程序设计', 'py_url': 'http://www.binhui.wang', 'text': <function text at 0x03A49108>}
```




■ 闭包closure

```
def deco():  
    name = "MING"  
    def wrapper():  
        print(name)  
    return wrapper
```

```
deco()()
```



■ 闭包案例

```
# 闭包函数，其中 exponent 称为自由变量
def nth_power(exponent):
    def exponent_of(base):
        return base ** exponent
    return exponent_of # 返回值是 exponent_of 函数
```

```
square = nth_power(2) # 计算一个数的平方
cube = nth_power(3)   # 计算一个数的立方
print(square(2))      # 计算 2 的平方
print(cube(2))        # 计算 2 的立方
```

■ 闭包的 `__closure__` 属性

闭包比普通的函数多了一个 `__closure__` 属性，该属性记录着自由变量的地址。当闭包被调用时，系统就会根据该地址找到对应的自由变量，完成整体的函数调用。

```
def nth_power(exponent):  
    def exponent_of(base):  
        return base ** exponent  
    return exponent_of  
square = nth_power(2)  
#查看 __closure__ 的值  
print(square.__closure__)
```

```
(<cell at 0x0000014454DFA948: int object at 0x00000000513CC6D0>,)
```



■ 闭包案例

```
def count_steps(original_steps=0):  
    def wrapper(new_steps):  
        nonlocal original_steps  
        total_steps = original_steps + new_steps  
        original_steps = total_steps  
        return original_steps  
    return wrapper  
  
count_steps = count_steps(8)  
print(count_steps(2))  
print(count_steps(3))  
print(count_steps(5))  
print(count_steps(5))
```



■ 改变作用域

■ 关键字: **global**

将 局部变量 变为全局变量

■ 关键字: **nonlocal**

可以在闭包函数中，引用并使用闭包外部函数的变量（非全局）



■ 改变作用域

```
def deco():  
    age = 10  
    def wrapper():  
        print(age)  
    return wrapper
```

```
deco()()
```

```
# 输出: 10
```



■ 改变作用域

```
def deco():  
    age = 10  
    def wrapper():  
        age += 1  
    return wrapper
```

```
deco()()
```




■ 改变作用域

```
def deco():  
    age = 10  
    def wrapper():  
        nonlocal age  
        age += 1  
    return wrapper
```

```
deco()()
```



■ global与nonlocal的区别

- 任何一层子函数，若直接使用全局变量且不对其改变的话，则共享全局变量的值；一旦子函数中改变该同名变量，则其降为该子函数所属的局部变量；
- global可以用于任何地方，声明变量为全局变量（声明时，不能同时赋值）；声明后再修改，则修改了全局变量的值；
- 而nonlocal的作用范围仅限于所在子函数的上一层函数中拥有的局部变量，必须在上层函数中已经定义过，且非全局变量，否则报错。

■ Python中的推导式

— 列表推导式List comprehensions

[表达式 **for** 迭代变量 **in** 可迭代对象 [**if** 条件表达式]]

[**if** 条件表达式]不是必须的，可以使用，也可省略

```
squares = [x**2 for x in range(10)]
```

— 字典/集合推导式

{表达式 **for** 迭代变量 **in** 可迭代对象 [**if** 条件表达式]}

■ 匿名函数

- **lambda表达式返回一个函数对象**

例子:

```
func = lambda x, y : x+y
```

func 相当于下面这个函数

```
def func(x,y):  
    return x+y
```

- **注意def是语句，而lambda是表达式**
- **`[(lambda x:x*x)(x) for x in range(1,11)]`**

■ 高阶函数 (Higher-order functions)

- A function is an instance of the Object type.
- You can store the function in a variable.
- You can pass the function as a parameter to another function.
- You can return the function from a function.
- You can store them in data structures such as hash tables, lists, ...

■ 高阶函数 (Higher-order functions)

- sorted ()
- map ()
- reduce ()
- filter ()

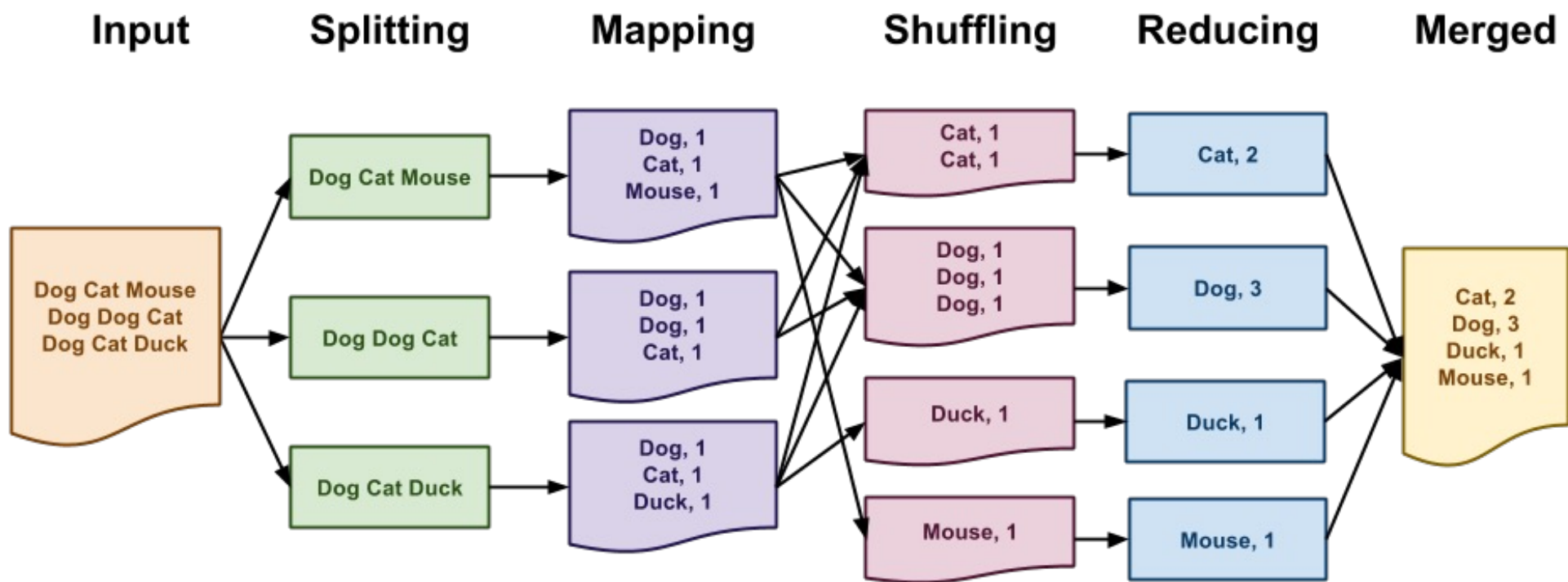
■ 高阶函数 (Higher-order functions)

- sorted ()

```
list_tuples = [(1, 'byd' ),  
               (3, 'xiaopeng' ),  
               (2, 'tesla' ),  
               (4, 'weilai')]
```

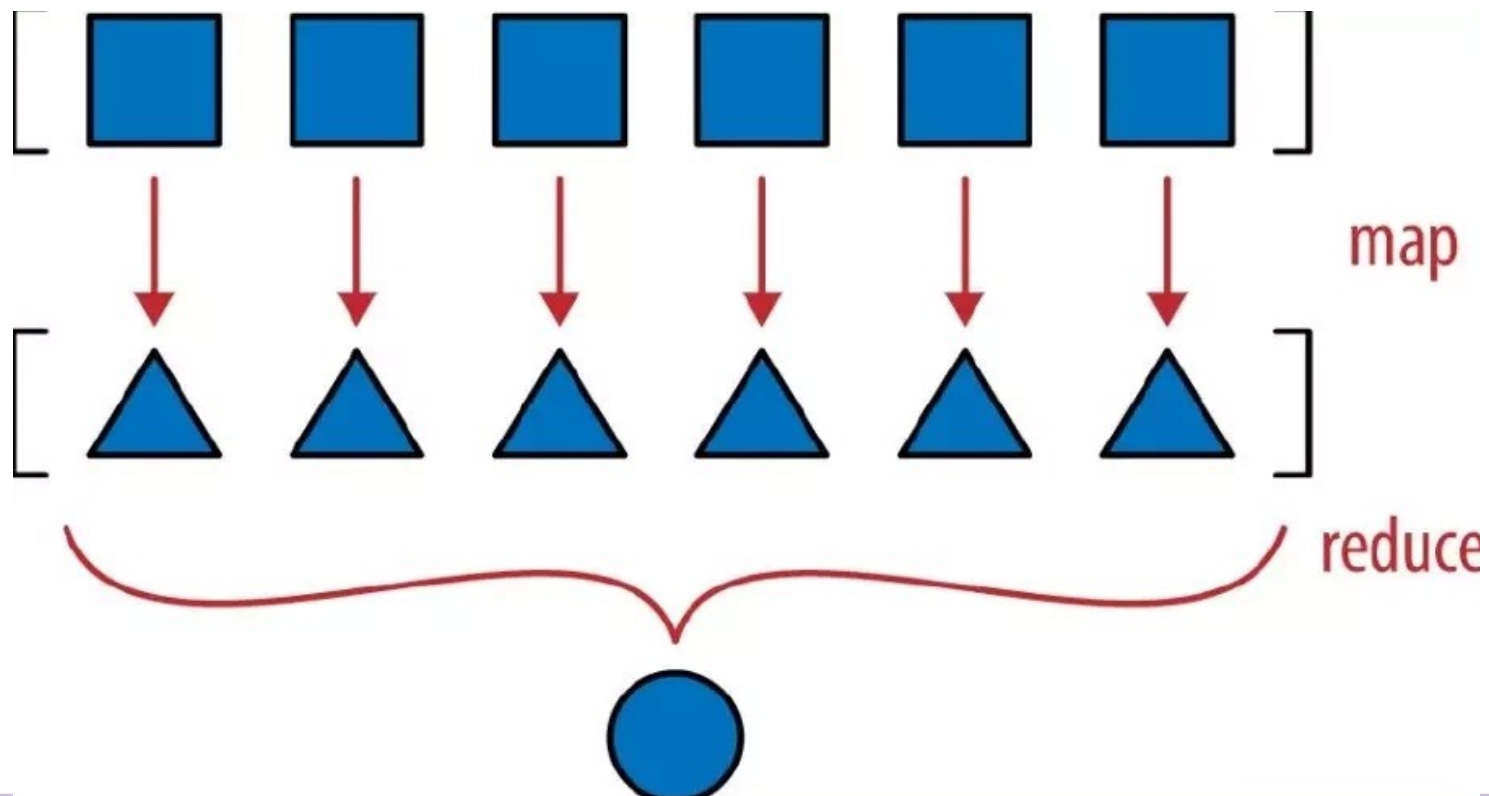
```
listed_tuples = sorted(list_tuples, key=lambda x: x[1])
```


■ 高阶函数 (Higher-order functions)



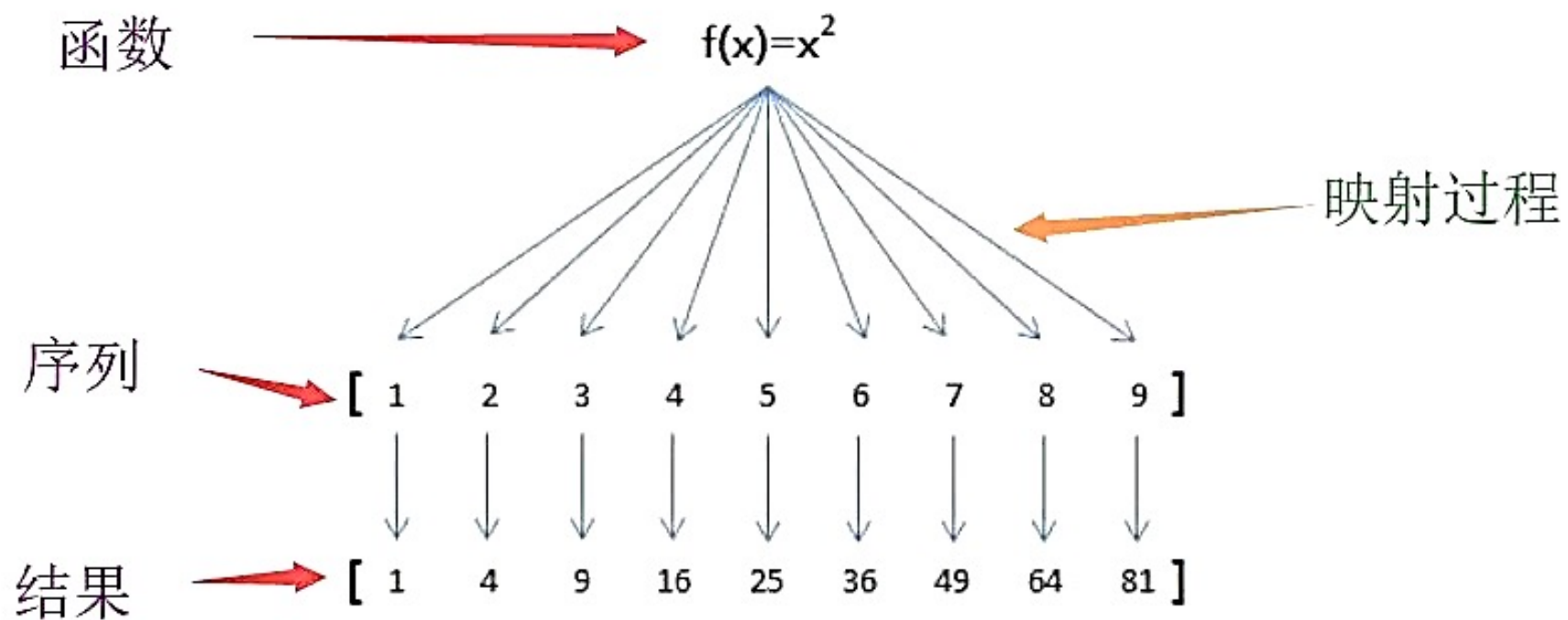
■ 高阶函数 (Higher-order functions)

— map、reduce



■ 高阶函数 (Higher-order functions)

— map





■ 高阶函数 (Higher-order functions)

- `map(function_to_apply, list_of_inputs)`
 - `function_to_apply`: 代表函数
 - `list_of_inputs`: 代表输入序列
- 注意: Python3中, `map`函数返回的是**迭代器**

■ 高阶函数 (Higher-order functions)

— map

```
items = [1, 2, 3, 4, 5] # 列表
squared = []
for i in items:
    squared.append(i**2)
```

■ 高阶函数 (Higher-order functions)

— map

```
items = [1, 2, 3, 4, 5]
```

```
def f(x):  
    return x ** 2
```

```
squared = list(map(f, items))
```

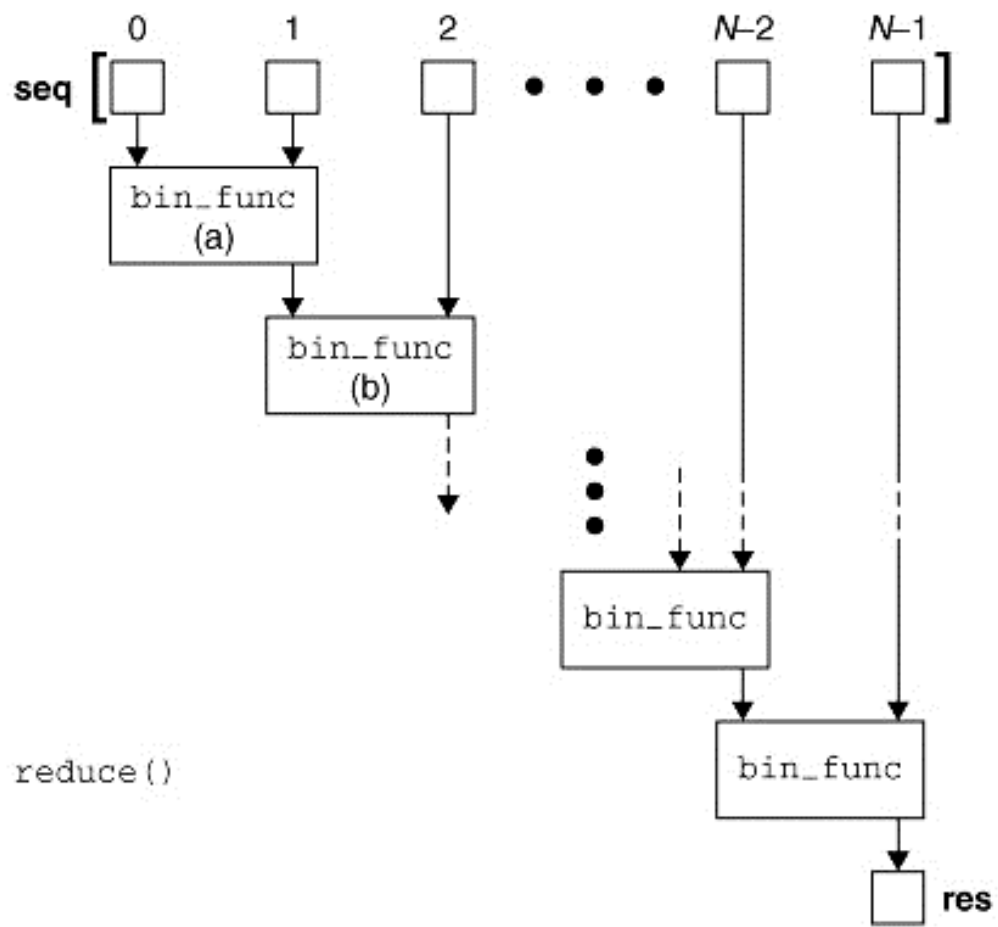
■ 高阶函数 (Higher-order functions)

— map

```
items = [1, 2, 3, 4, 5]  
squared = list(map(lambda x: x**2, items))
```


■ 高阶函数 (Higher-order functions)

— reduce



(a) The value of this result is `bin_func(seq[0], seq[1])`

(b) The value of this result is `bin_func(bin_func(seq[0], seq[1]), seq[2])`, etc.



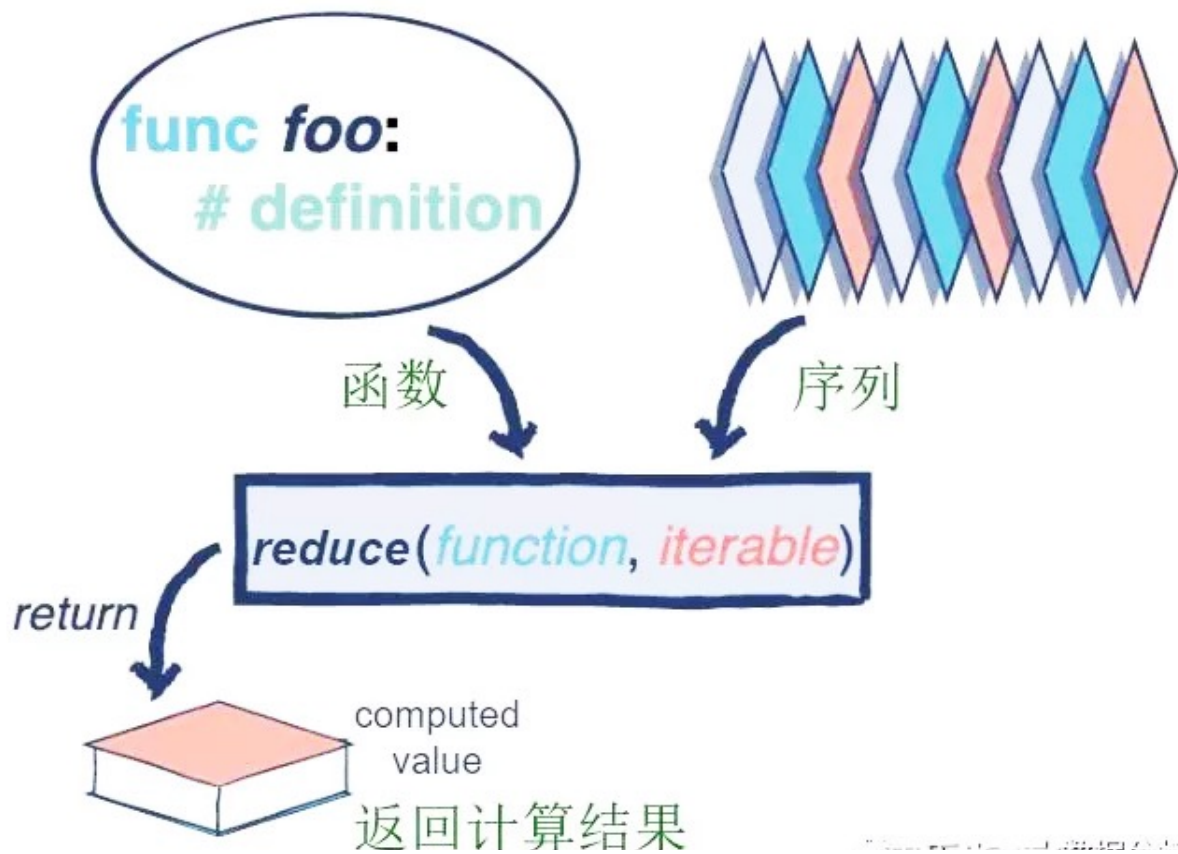
■ 高阶函数 (Higher-order functions)

— `reduce(function, iterable[, initializer])`

- **function:** 代表函数
- **iterable:** 序列
- **initializer:** 初始值 (可选)
- 与map不同, `reduce`需要用`from functools import reduce`导入, 才可使用

■ 高阶函数 (Higher-order functions)

— reduce





■ 高阶函数 (Higher-order functions)

— reduce

求10的阶乘

导入reduce

```
from functools import reduce
```

定义函数

```
def f(x, y):  
    return x * y
```

定义序列，含1~10的元素

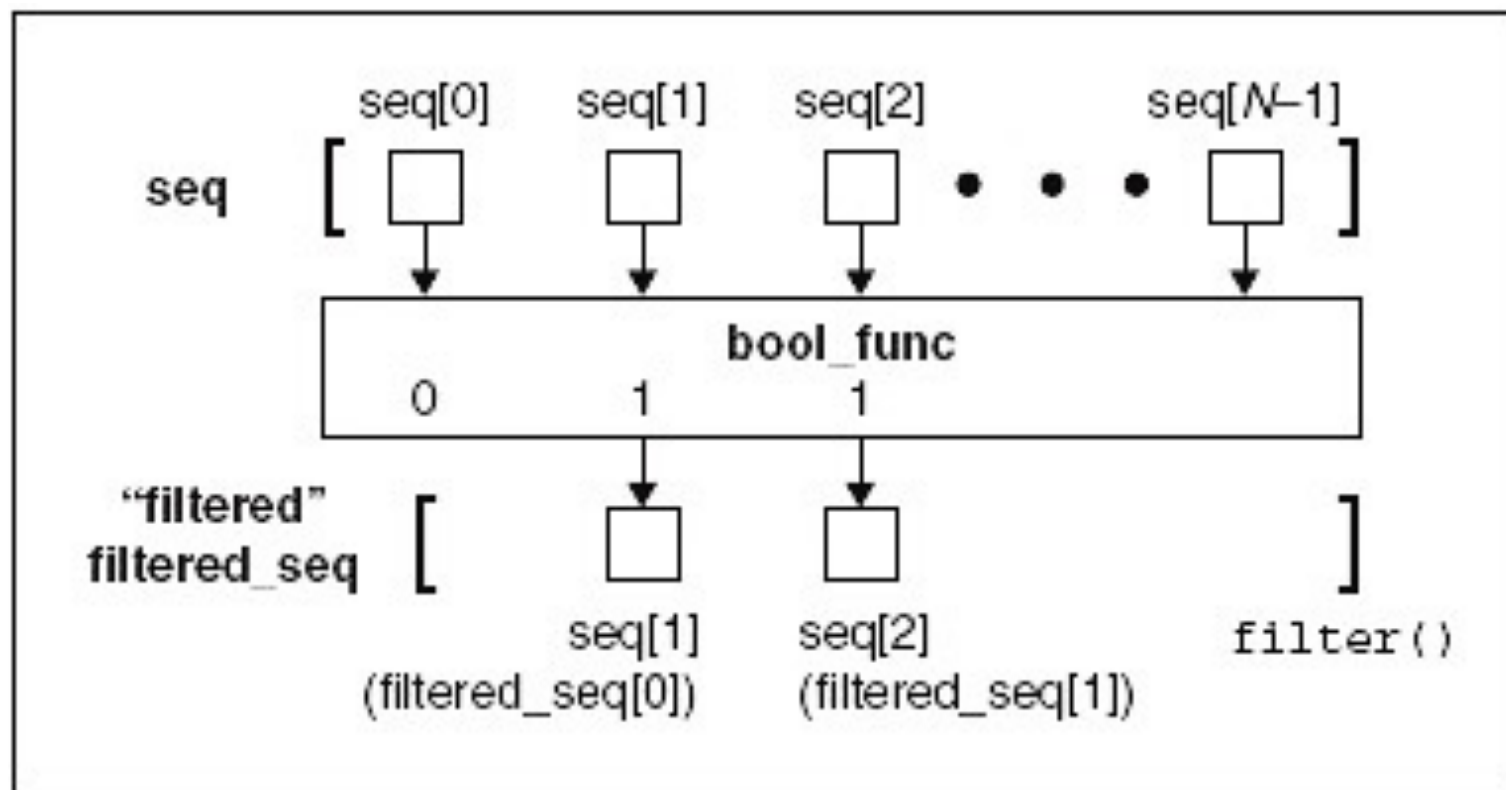
```
items = range(1, 11)
```

使用reduce方法

```
result = reduce(f, items)  
print(result)
```

■ 高阶函数 (Higher-order functions)

— filter



■ 高阶函数 filter ()

过滤出列表中的所有奇数

```
def is_odd(n):  
    return n % 2 == 1
```

```
f_list = filter(is_odd, [1, 2, 3, 4, 5, 6, 7, 8, 9, 10])  
print(list(f_list))
```



■ 偏函数 `partial()`

```
int2 = partial(int, base=2)  
print(int2('111000111'))
```